

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ**

**«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ им. А.Н. Косыгина
(ТЕХНОЛОГИИ. ДИЗАЙН. ИСКУССТВО)»
(ФГБОУ ВО «РГУ им. А.Н. Косыгина»)**

Семенов А.А.

**РАЗРАБОТКА НА ПЛАТФОРМЕ 1С:ПРЕДПРИЯТИЕ 8.3.
ЧАСТЬ 2**

Учебное пособие

*Допущено к изданию редакционно-издательским
советом университета в качестве учебного пособия для подготовки
бакалавров по направлениям 09.03.01 Информатика и вычислительная
техника, 09.03.02 Информационные системы и технологии*

Москва
ФГБОУ ВО «РГУ им. А.Н. Косыгина», 2024

УДК [657:004.4](075)
С 30

С 30 Семенов А.А.

Разработка на платформе 1С:Предприятие 8.3. Часть 2: Учебное пособие –
М.: ФГБОУ ВО «РГУ им. А.Н. Косыгина», 2024. – 48 с.

Рецензенты:

Радченко И.А. - директор ООО "АВК-Системс";

Литвин Е.В. - доцент ФГБОУ ВО «РГУ им. А.Н. Косыгина», канд.
техн. наук.

Учебное пособие предназначено для обучающихся по направлениям подготовки 09.03.01 Информатика и вычислительная техника, 09.03.02 Информационные системы и технологии очной формы обучения и будет использовано при изучении дисциплины "Технология разработки прикладных решений для системы 1С:Предприятие".

Пособие поможет студентам в освоении популярного в России программного пакета 1С:Предприятие 8.3 на уровне разработчика. Материал содержит теоретические и практические сведения, необходимые при разработке специализированных прикладных решений 1С. В ходе рассмотрения и реализации практических примеров разработки даются рекомендации по работе с конфигуратором и программированием в среде 1С версии 8.3.

УДК [657:004.4](075)

Подготовлено к печати на кафедре
информационных технологий.

Печатается в авторской редакции.

ОГЛАВЛЕНИЕ

Введение	4
Постановка задачи	4
Создание новой команды «Замена цвета»	6
Создание новой команды «Замена страны»	11
Создание новой команды «Создать группу»	14
Создание команды «Выгрузить в текстовый файл»	18
Создание команды «Загрузить из текстового файла»	22
Создание команды «Удалить товар»	27
Создание команды «Список файлов»	31
Создание команды «Сжатие файлов»	34
Создание команды «Распаковка архива»	36
Создание команды «Новый документ»	38
Создание команды «Провести документы»	40
Создание команды «Перенос из одной группы в другую»	46
Список литературы	47

Введение

Данное учебное пособие является продолжением вышедшего ранее учебного пособия «Разработка на платформе 1С:Предприятие 8.3. Часть 1» [7]. Целью серии этих учебных пособий, посвященных разработке на платформе 1С, является формирование практических представлений о работе с основными объектами и механизмами этой платформы. Все теоретические и практические сведения, раскрытые в 1-й части, здесь будут опущены. В этой части основное внимание будет уделено обработкам. В отличие от отчетов, обработки освещаются, как правило, менее подробно в справочных материалах по 1С, поэтому в данном учебном пособии постараемся восполнить этот пробел. К тому же, при создании обработок хорошо нарабатываются знания и навыки по программированию в среде 1С.

Обработки в 1С — это прикладные объекты конфигурации, предназначенные для выполнения различных действий над данными в информационной базе, иначе для обработки данных. Посредством обработок можно выполнять различные операции над данными, например удаление из системы устаревших данных, модификацию данных, импорт данных из других систем, экспорт данных в другие системы и т.д. Характер выполняемых в этом случае действий отражает название объекта конфигурации — «Обработка», так как в результате информация, хранящаяся в системе, претерпевает какие-либо изменения. Этим обработки кардинально отличаются от отчетов, призванных для обработки накопленной информации и получения сводных данных в удобном для просмотра и анализа виде [8].

Постановка задачи

За основу возьмём разработанное в 1-й части [7] прикладное решение 1С, позволяющее автоматизировать простую складскую деятельность некоторой вымышленной торговой фирмы. На практических примерах рассмотрим ряд обработок, раскрывающих базовые вопросы обработки данных в информационных базах 1С.

Первым делом создадим в нашей конфигурации новую обработку. Для этого в дереве метаданных найдем объект «Обработки» и выберем «Добавить». Зададим имя и синоним для нашей обработки. Далее на вкладке «Формы» добавим новую форму обработки, нажав кнопку с

пиктограммой лупы (см. рис. 1). В появившемся конструкторе формы обработки оставим всё по умолчанию и нажмём кнопку «Готово».

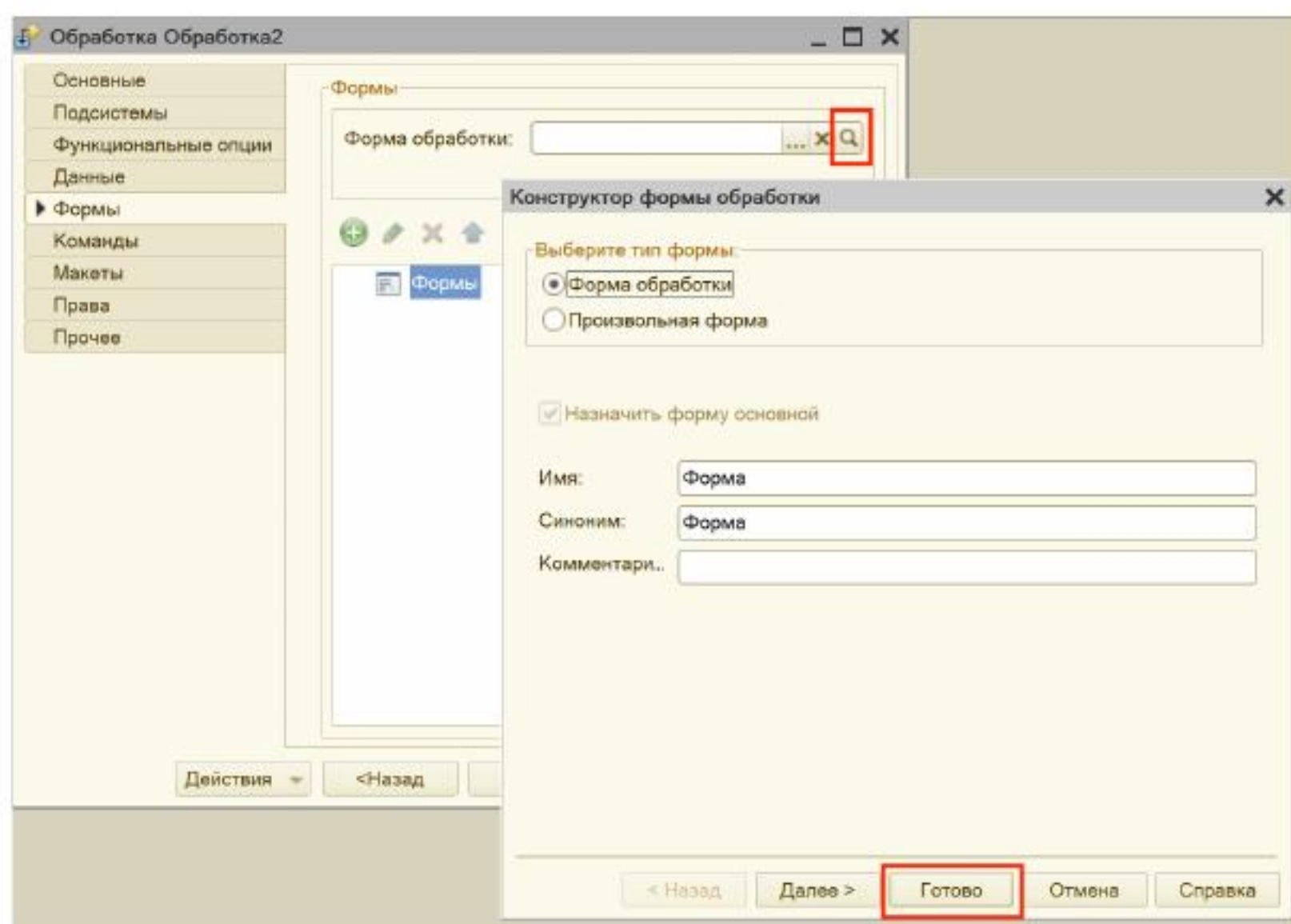


Рис. 1. Создание новой формы обработки

Перед нами появится конфигуратор формы, представленный на рисунке 2. Как видим, внизу имеются две вкладки «Форма» и «Модуль». Вкладка «Форма» предназначена для управления визуальными компонентами, размещаемыми на форме обработки, а вкладка «Модуль» — для создания и редактирования процедур/функций, иными словами для работы с программным кодом обработки.

Необходимо отметить, что при разработке форм объектов конфигурации разработчик не имеет возможности нарисовать форму. Он может только указать из каких элементов будет состоять форма, а система уже сама автоматически расположит эти элементы на управляемой форме, конечно, если речь идёт об интерфейсе «Такси». Элементы формы в верхнем левом окне редактора форм образуют иерархическую структуру, из которой следует, что чем выше в списке находится элемент, тем выше и левее на форме он будет располагаться. [1]

Эта структура редактируется на закладке «Элементы» и позволяет управлять отображением и редактированием данных. Результат изменений

сразу отразится в нижней части редактора форм. При этом разработчику не нужно задумываться над вопросами попиксельного позиционирования того или иного элемента (компонента) формы, его размеров и привязки к другим элементам, так как за это отвечает система.

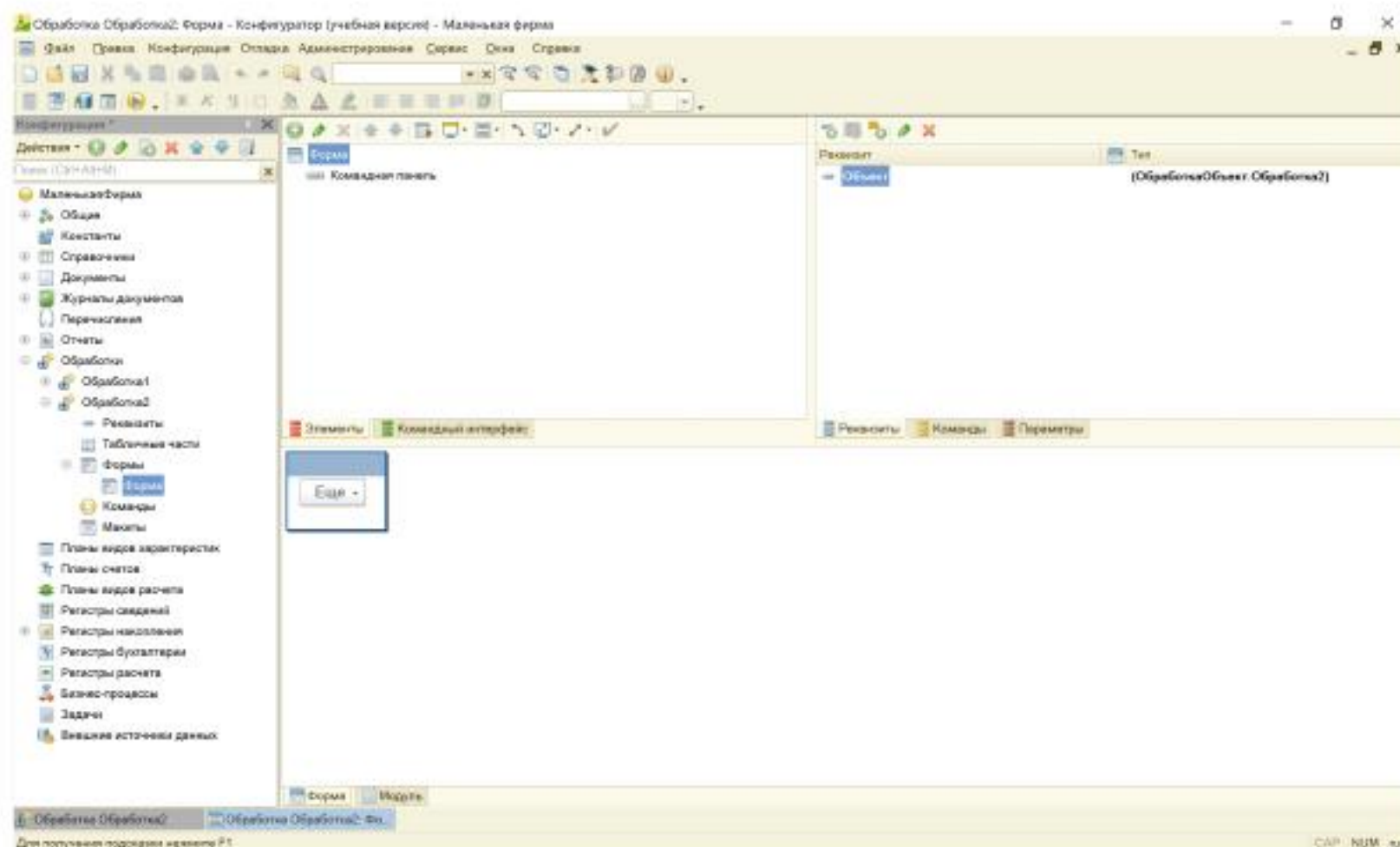


Рис. 2. Конфигуратор формы нашей обработки (Окно редактора форм)

Создание новой команды «Замена цвета»

Создадим обработку, которая по команде «Замена цвета» будет находить все товары зелёного цвета и только для них заменять цвет на черный. На самом деле, подобные задачи часто встречаются в реальной жизни. Например, бывают такие ситуации, когда были допущены ошибки при создании номенклатуры товаров, состоящей из десятков тысяч наименований, и необходимо оперативно эти ошибки устранить. В подобных ситуациях часто прибегают к созданию специальных обработок, позволяющих оперативно исправить различные ошибки в информационной базе 1С.

Итак, в правой верхней части окна редактора форм выберем вкладку «Команды», затем «Команды формы». Добавим новую команду с именем «ЗаменаЦвета». Далее, в поле «Действие» нажмем на кнопку с пиктограммой лупы, после чего перед нами появится окно выбора варианта создания обработчика команды (см. рис. 3).

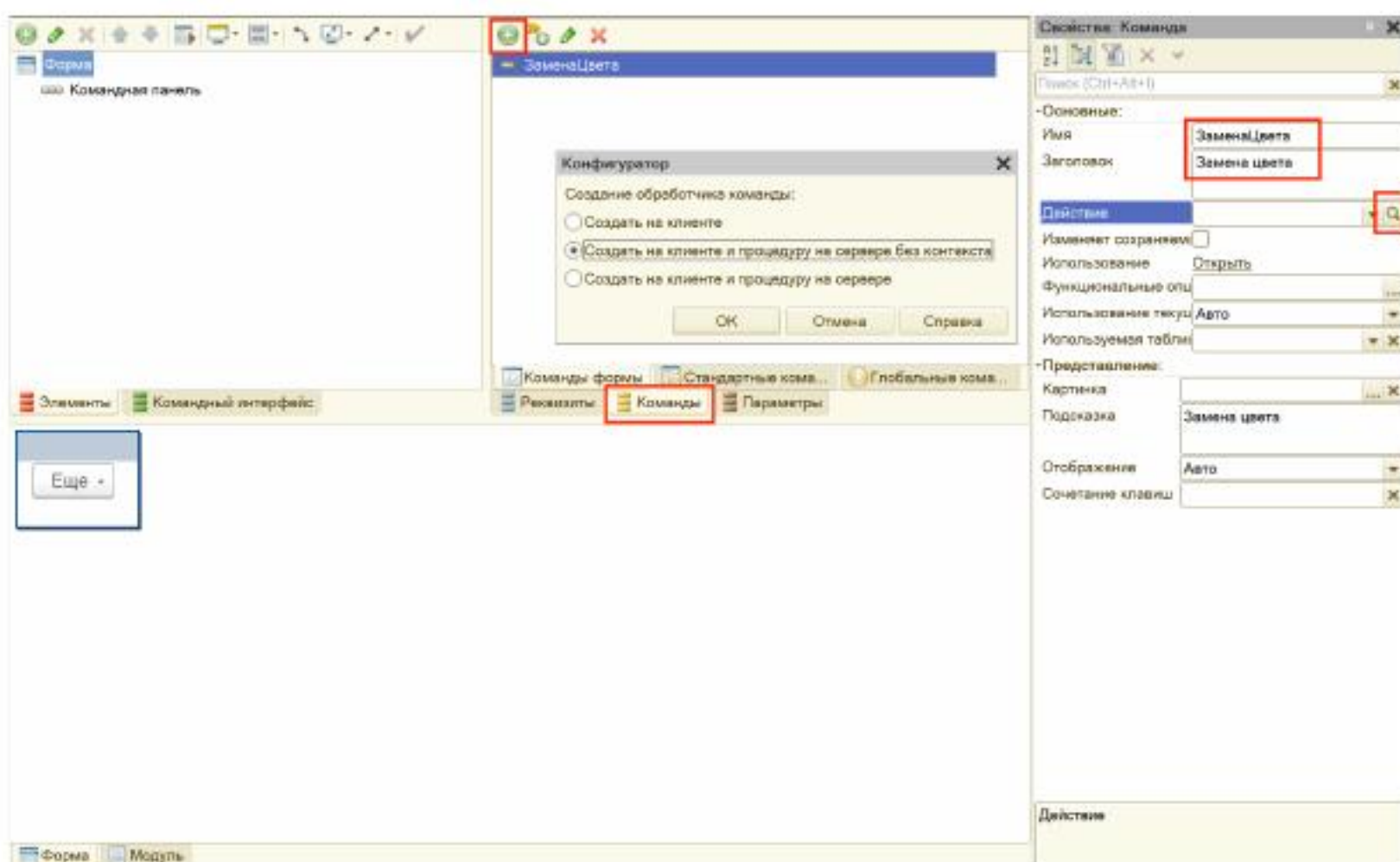


Рис. 3. Создание команды «ЗаменаЦвета»

Предлагается выбрать один из следующих вариантов директив препроцессора: 1) создать на клиенте; 2) создать на клиенте и процедуру на сервере без контекста; 3) создать на клиенте и процедуру на сервере.

Первый вариант нам не подходит, так как он не позволит обратиться к базе данных, а нам в рамках нашей задачи доступ к базе данных необходим. Третий вариант слишком избыточен для нас, так как мы не собираемся работать с контекстом формы. Выберем второй вариант «Создать на клиенте и процедуру на сервере без контекста» (см. рис. 3). При этом варианте процедура будет выполняться на сервере, вне контекста формы. Т.е. процедура не будет иметь доступа к реквизитам формы и не может к ним обращаться ни для чтения, ни для изменения, программный код будет выполняться на сервере и возвращать желаемый результат. При этом варианте не происходит отправка на сервер всей XML-разметки формы и значений реквизитов формы, что в значительной степени экономит ресурсы и ускоряет взаимодействие с сервером. Этот вариант хорошо подходит для получения статичных данных, когда не требуется работать с формой и значениями реквизитов формы и крайне рекомендуется к использованию для оптимизации клиент-серверного взаимодействия.

После выбора нужного нам варианта директивы препроцессора, система создаст заготовку программного кода с обработчиком события для нашей команды. Выберем вкладку «Модуль» и увидим там следующий код.

```
&НаСервереБезКонтекста  
Процедура ЗаменаЦветаНаСервере()  
    // Вставить содержимое обработчика.  
КонецПроцедуры
```

```
&НаКлиенте  
Процедура ЗаменаЦвета(Команда)  
    ЗаменаЦветаНаСервере();  
КонецПроцедуры
```

Вместо комментария «// Вставить содержимое обработчика» необходимо написать программный код, который будет решать нашу задачу. Реализуем следующую логику.

Вначале, с помощью метода «Выбрать» сформируем выборку всех элементов нашего справочника «Товары» и запишем её в переменную «ВыборкаТоваров».

```
ВыборкаТоваров = Справочники.Товары.Выбрать();
```

Далее, в цикле «Пока есть следующий элемент» будем перебирать все элементы из нашей выборки и, если у товара зеленый цвет, будем менять его на красный. Но здесь есть нюанс, который заключается в том, что тип данных у реквизита «ЦветТовара» – справочник «ЦветаТоваров», а не просто строка. Поэтому, меняя цвета для товара, необходимо оперировать не просто строковыми названиями цветов, а именно элементами справочника «ЦветаТоваров». В итоге у нас получится следующий программный код.

```
&НаСервереБезКонтекста  
Процедура ЗаменаЦветаНаСервере()  
    ВыборкаТоваров = Справочники.Товары.Выбрать();  
    Пока ВыборкаТоваров.Следующий() Цикл  
        Если СокрЛП(ВыборкаТоваров.ЦветТовара) = "Зеленый"  
Тогда
```



```

        ТекущийТовар                                     =
ВыборкаТоваров.Ссылка.ПолучитьОбъект();
        КрасныйЦвет                                       =
Справочники.ЦветаТоваров.НайтиПоКоду("Красный");
        ТекущийТовар.ЦветТовара = КрасныйЦвет;
        ТекущийТовар.Записать();
        Сообщить("Для товара " + ТекущийТовар.Наименование
+ " заменен зеленый цвет на красный.");

```

КонецЕсли;

КонецЦикла;
КонецПроцедуры

&НаКлиенте
Процедура ЗаменаЦвета(Команда)
ЗаменаЦветаНаСервере();
КонецПроцедуры

Расшифруем некоторые функции/процедуры/методы, которые использовали в цикле.

СокрЛП(<Строка>) - отсекает незначащие символы, стоящие слева от первого значащего символа в строке, и пробелы, стоящие справа от последнего значащего символа в строке. Таким образом, используя СокрЛП(ВыборкаТоваров.ЦветТовара), мы получаем строковое значение цвета, причем с отсеченными незначащими символами, которые операторы информационной базы 1С могли случайно ввести при заполнении карточек цветов товаров, например: пробел, неразрывный пробел, табуляция и т.п.

Ссылка – получаем ссылку на элемент справочника.
ПолучитьОбъект() – получаем по ссылке объект, предназначенный для чтения, изменения, добавления и удаления справочника.

НайтиПоКоду(<Код>, <ПоискПоПолномуКоду>, <Родитель>, <Владелец>) – осуществляет поиск элемента по его коду. Напомним, что когда мы создавали справочник «ЦветаТоваров», мы заменили код на наименование цвета и сделали тип кода строкой и настроили основное представление в виде кода. Поэтому строка кода

Справочники.ЦветаТоваров.НайтиПоКоду("Красный") найдет и вернет нам элемент справочника «ЦветаТоваров» с кодом «Красный».

Записать() – записывает элемент справочника в базу данных.

После написания рассмотренного выше программного кода, создадим кнопку «Замена цвета» на форме нашей обработки. Для этого переключимся на вкладку «Форма» и перетащим нашу команду «ЗаменаЦвета» в левую часть элементов формы. После этого, произойдет создание и размещение кнопки на форме (см. рис. 4).

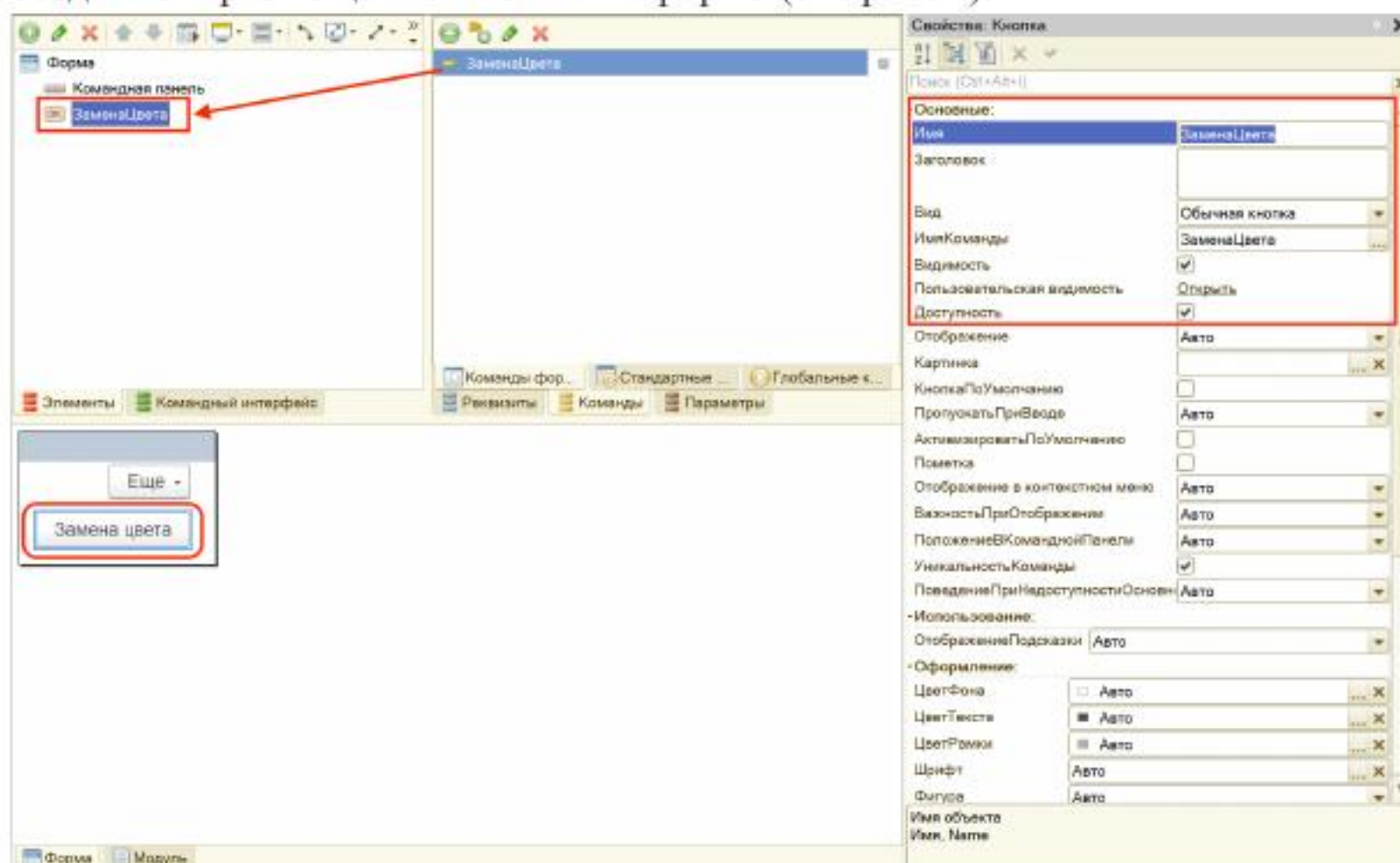


Рис. 4. Создание кнопки «Замена цвета» на форме

При необходимости можно задать для кнопки другой заголовок, выбрать вид, а также настроить видимость, доступность и многое другое (см. рис. 4).

Запустим теперь 1С в пользовательском режиме. Для этого нажмем F5 или кликнем по кнопке «Начать отладку». Выберем в меню нашу обработку и после её запуска на её форме нажмем на нашу кнопку «Замена цвета». В результате её работы, для всех товаров зеленого цвета, цвет будет заменен на красный (см. рис. 5). Таким образом, поставленная задача решена.

Для проверки создайте в справочнике «Товары» несколько товаров зеленого цвета и несколько товаров других цветов. В результате запуска данной обработки только для товаров зеленого цвета должна произойти замена цвета на красный.

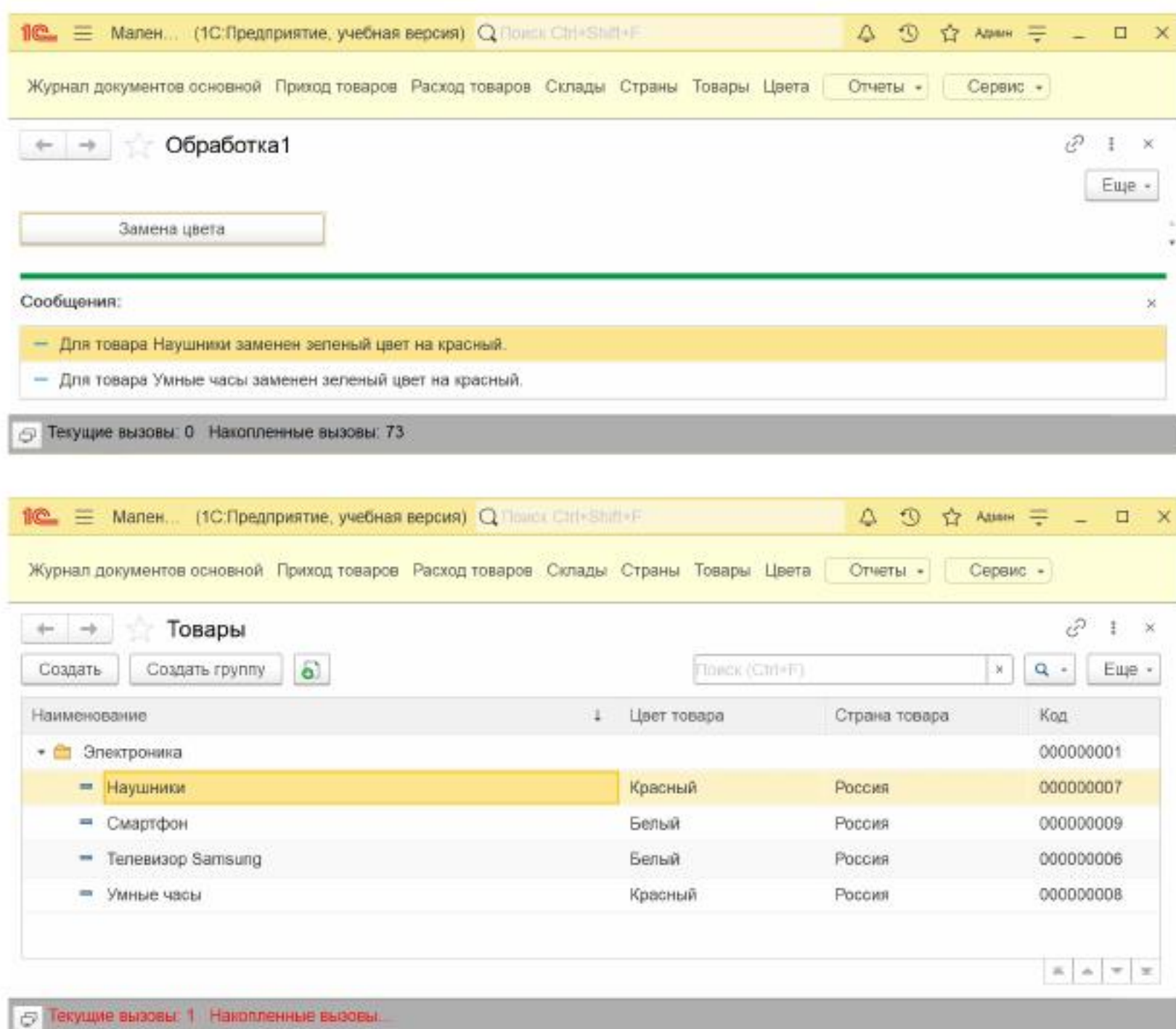


Рис. 5. Результат работы обработки «Замена цвета»

Создание новой команды «Замена страны»

По аналогии с предыдущим примером, создадим обработку, которая по команде «Замена страны» будет находить в определенной группе (категории), например «Электроника», все товары китайского производства и менять для них страну производства на Россию.

Создадим команду «ЗаменаСтраны» и выполним аналогично предыдущему примеру все необходимые действия. Единственное что будет отличаться, так это программный код, а он будет следующим.

&НаСервереБезКонтекста

Процедура ЗаменаСтраныНаСервере()


```

Категория = "Электроника"; // строка
//категория как объект
ГруппаТоваров =
Справочники.Товары.НайтиПоНаименованию(Категория);
// выбираются товары у которых Родитель - "Электроника"
ВыборкаТоваров = Справочники.Товары.Выбрать(ГруппаТоваров);
СтранаКитай =
Справочники.СтраныТоваров.НайтиПоНаименованию("Китай");
Пока ВыборкаТоваров.Следующий() Цикл
    Если ВыборкаТоваров.СтранаТовара = СтранаКитай Тогда
        ТекущийТовар =
ВыборкаТоваров.Ссылка.ПолучитьОбъект();
        СтранаРоссия =
Справочники.СтраныТоваров.НайтиПоНаименованию("Россия");
        ТекущийТовар.СтранаТовара = СтранаРоссия;
        ТекущийТовар.Записать();
        Сообщить("Для товара " + ТекущийТовар.Наименование
+ " заменена страна.");
    КонецЕсли;
КонецЦикла;
КонецПроцедуры

&НаКлиенте
Процедура ЗаменаСтраны(Команда)
    ЗаменаСтраныНаСервере();
КонецПроцедуры

```

Разберем закодированную логику. Вначале указываем наименование категории, в которой собираемся делать замену: *Категория* = "Электроника";. Далее получаем объект заданной категории с помощью поиска по наименованию и записываем его в переменную *ГруппаТоваров*: *ГруппаТоваров* = *Справочники.Товары.НайтиПоНаименованию(Категория)*;

рия);. Затем записываем в переменную ВыборкаТоваров все товары, у которых реквизит «Родитель» соответствует нашей категории: *ВыборкаТоваров = Справочники.Товары.Выбрать(ГруппаТоваров);*. Это мы делаем с помощью метода «Выбрать» с параметром ГруппаТоваров, что обеспечивает отбор товаров только из заданной категории. Далее получаем объект из справочника «СтраныТоваров», у которого наименование «Китай»: *СтранаКитай = Справочники.СтраныТоваров.НайтиПоНаименованию("Китай");*. Записываем его в переменную СтранаКитай. Затем перебираем в цикле все товары из нашей выборки и если у какого-то из них значение реквизита «СтранаТовара» - Китай, то получаем по ссылке объект этого товара и меняем страну с последующей записью изменений и выводом сообщения о том, что страна для данного товара изменена.

Разместим кнопку на форме (см. рис. 6).

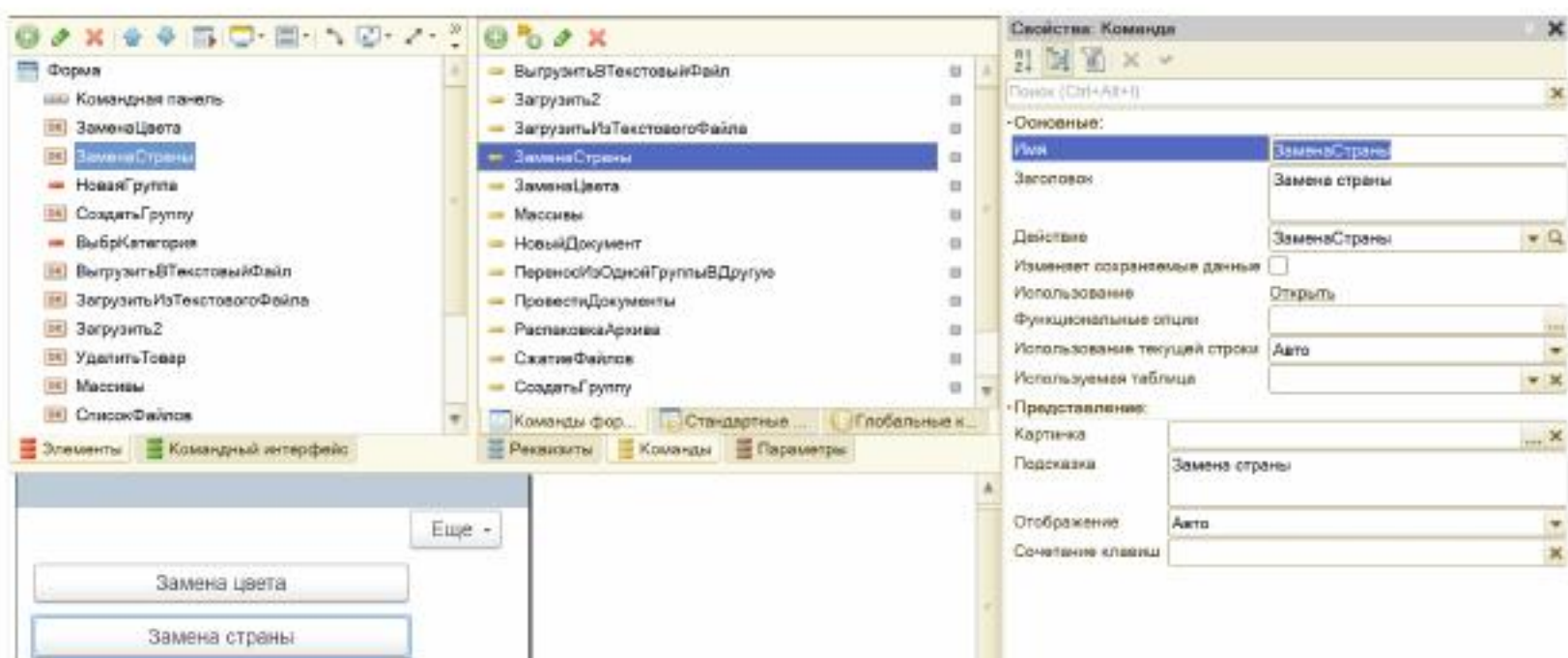


Рис. 6. Создание кнопки «Замена страны» на форме

Запустим 1С в пользовательском режиме, подтвердив применение всех изменений. Для проверки данной обработки, создадим в справочнике «Товары» в категории «Электроника» несколько товаров, страна производства которых Китай. В результате запуска данной обработки только для товаров китайского производства и только в категории «Электроника» должна произойти замена страны (см. рис. 7). Как видно из данного скриншота, всё именно так и произошло.

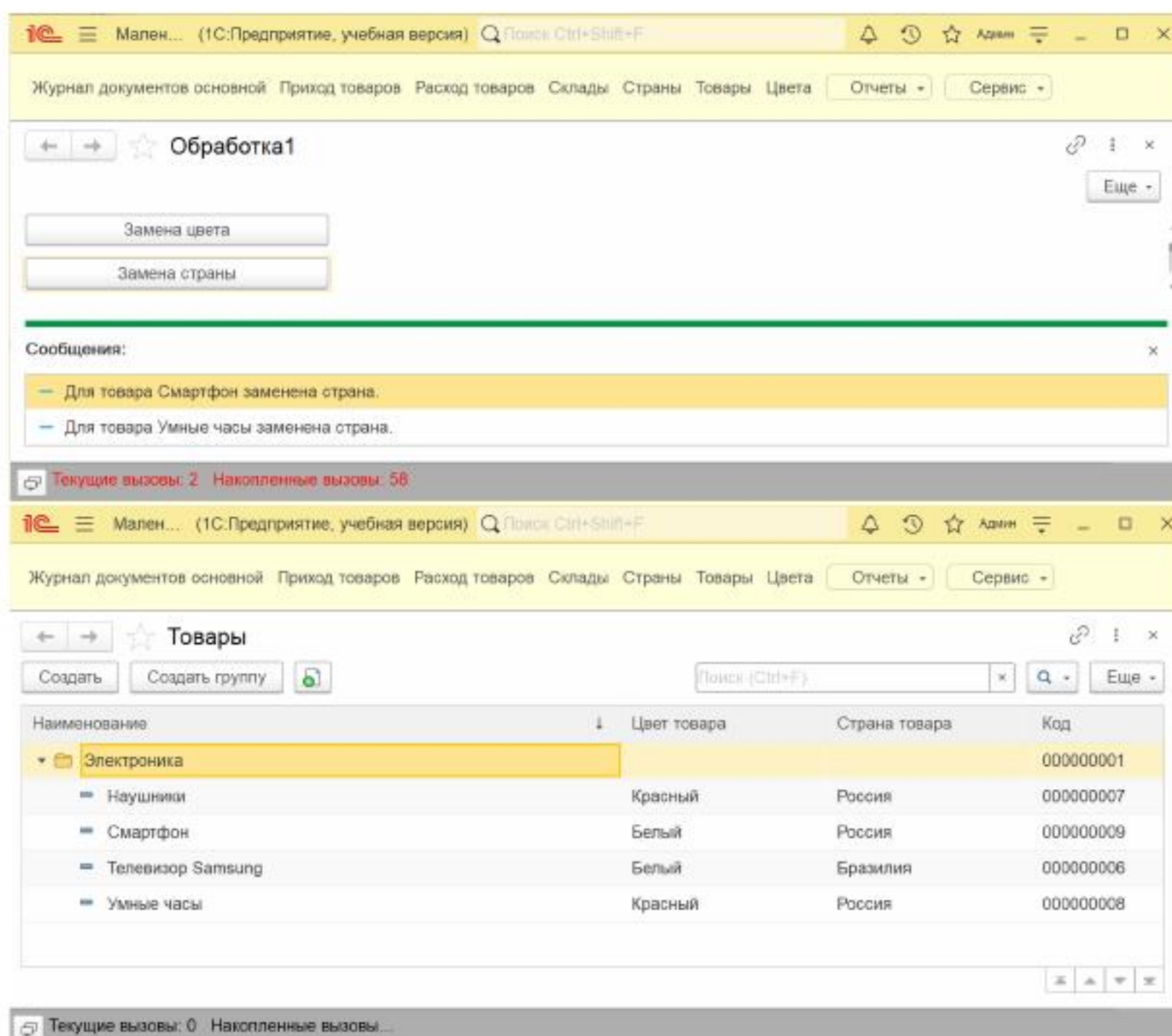


Рис. 7. Результат работы обработки «Замена страны»

Создание новой команды «Создать группу»

Создадим обработку, которая будет предоставлять пользователю возможность ввести имя новой категории товаров и создавать её. Для этого в верхней правой части окна редактора форм перейдем на вкладку «Реквизиты» и добавим новый реквизит «НоваяГруппа». В его свойствах, для свойства «Тип» укажем Строка (см. рис. 8). Таким образом, у этого реквизита будет строковый тип данных.

Далее перетащим этот реквизит в левую верхнюю часть окна редактора элементов, располагающихся на форме. В результате этого на нашей форме отобразится поле ввода, в которое пользователь сможет вводить наименование новой создаваемой категории (см. рис. 9).

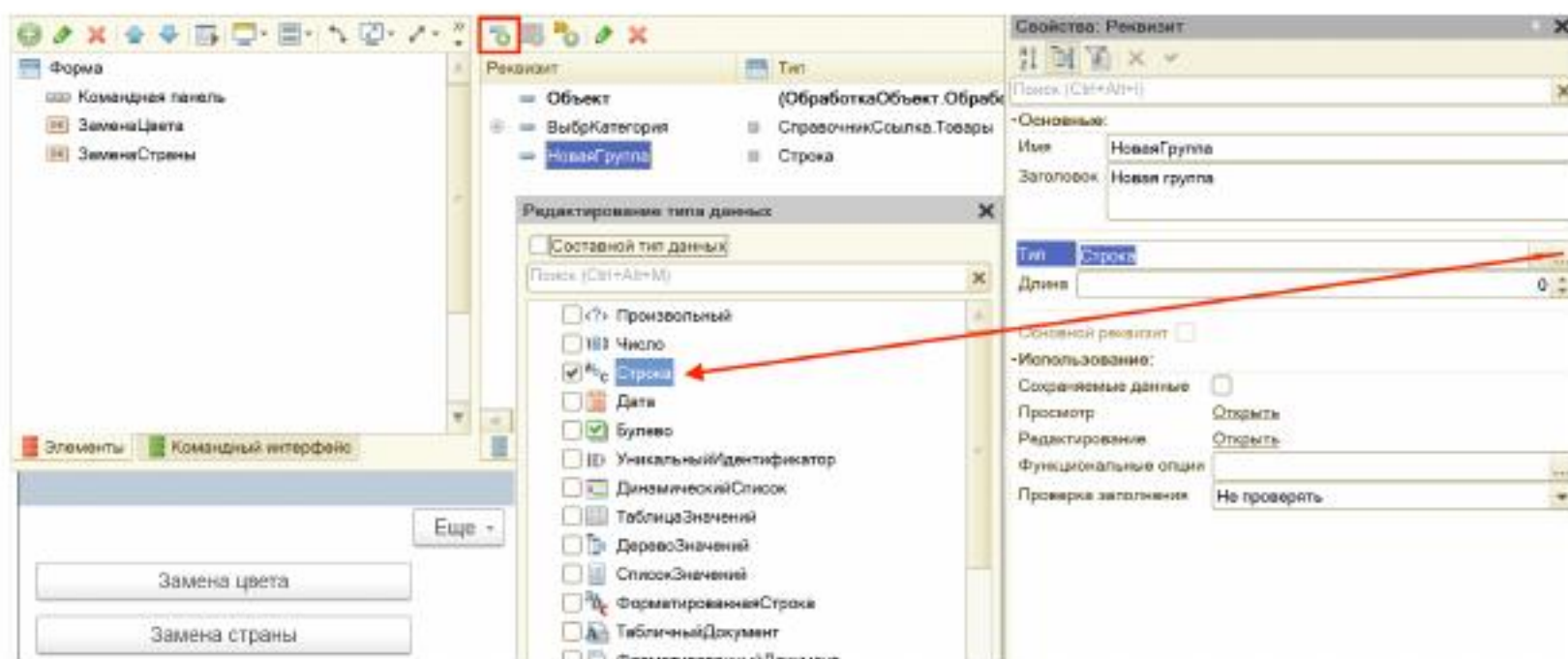


Рис. 8. Добавление реквизита «НоваяГруппа»

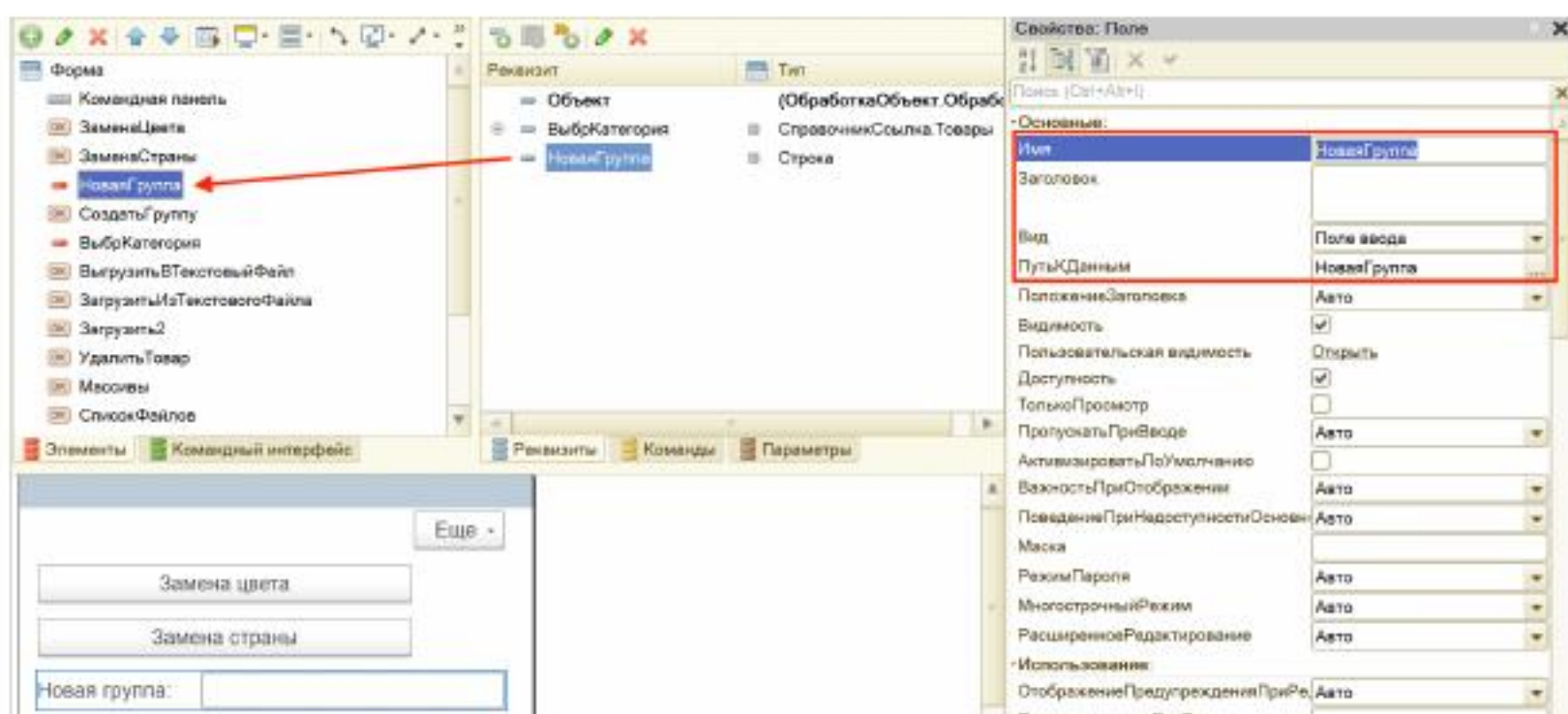


Рис. 9. Добавление на форму поля ввода «Новая группа»

После этого, по аналогии с предыдущими примерами, создадим команду «Создать группу». Программный код команды будет следующим.

&НаСервереБезКонтекста

Процедура СоздатьГруппуНаСервере(НовГруппа)

НоваяКатегория = Справочники["Товары"].СоздатьГруппу();

НоваяКатегория.Наименование = СокрЛП(НовГруппа);

НоваяКатегория.Записать();

НаимКатегории = СокрЛП(НовГруппа);

Товары = Справочники.Товары;


```

        НайденнаяСсылка =
Товары.НайтиПоНаименованию(НаимКатегории);
        Если НайденнаяСсылка = Товары.ПустаяСсылка() Тогда
            Сообщить("Категория '" + НаимКатегории + "' не создана!");

        Иначе
            Сообщить("Категория '" + НаимКатегории + "' успешно
создана!");

        КонецЕсли;

КонецПроцедуры

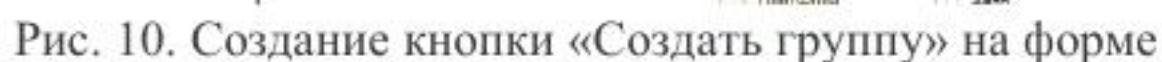
&НаКлиенте
Процедура СоздатьГруппу(Команда)
    СоздатьГруппуНаСервере(НоваяГруппа);
КонецПроцедуры

```

Рассмотрим данный программный код подробнее. Вначале на клиенте будет вызываться процедура СоздатьГруппу(Команда), которая в свою очередь будет запускать на сервере процедуру СоздатьГруппуНаСервере(НоваяГруппа). Эта процедура вызывается с аргументом НоваяГруппа, который как раз и содержит название категории, которое ввёл пользователь на форме нашей обработки. Таким образом, введенное пользователем наименование группы (категории товаров) передается в процедуру, выполняющуюся на сервере, логика работы которой следующая.

Значение аргумента НоваяГруппа на сервере мы принимаем в параметре НовГруппа. С помощью метода СоздатьГруппу(), создаем новую группу справочника «Товары» и записываем её в переменную НоваяКатегория. Далее в строке *НоваяКатегория.Наименование = СокрЛП(НовГруппа)* присваиваем свойству «Наименование» нашей новой категории строковое значение введенного пользователем наименования группы без незначащих символов. Затем для нашей новой категории вызываем метод Записать(), за счет чего происходит добавление новой категории (группы) в справочник «Товары». Далее производим проверку

Проверим код на ошибки посредством функции «Проверка модуля» (Ctrl+F7) и разместим кнопку для этой команды на форме (см. рис. 10).



Мален... (1С:Предприятие, учебная версия) Поиск Ctrl+Shift+F

Журнал документов основной Приход товаров Расход товаров Склады Страны Товары Цвета Отчеты Сервис

← → ☆ Обработка1

Замена цвета

Замена страны

Новая группа: Медицинская техника

Создать группу

Сообщения:

— Категория 'Медицинская техника' успешно создана!

Текущие вызовы: 2 Накопленные вызовы: 25

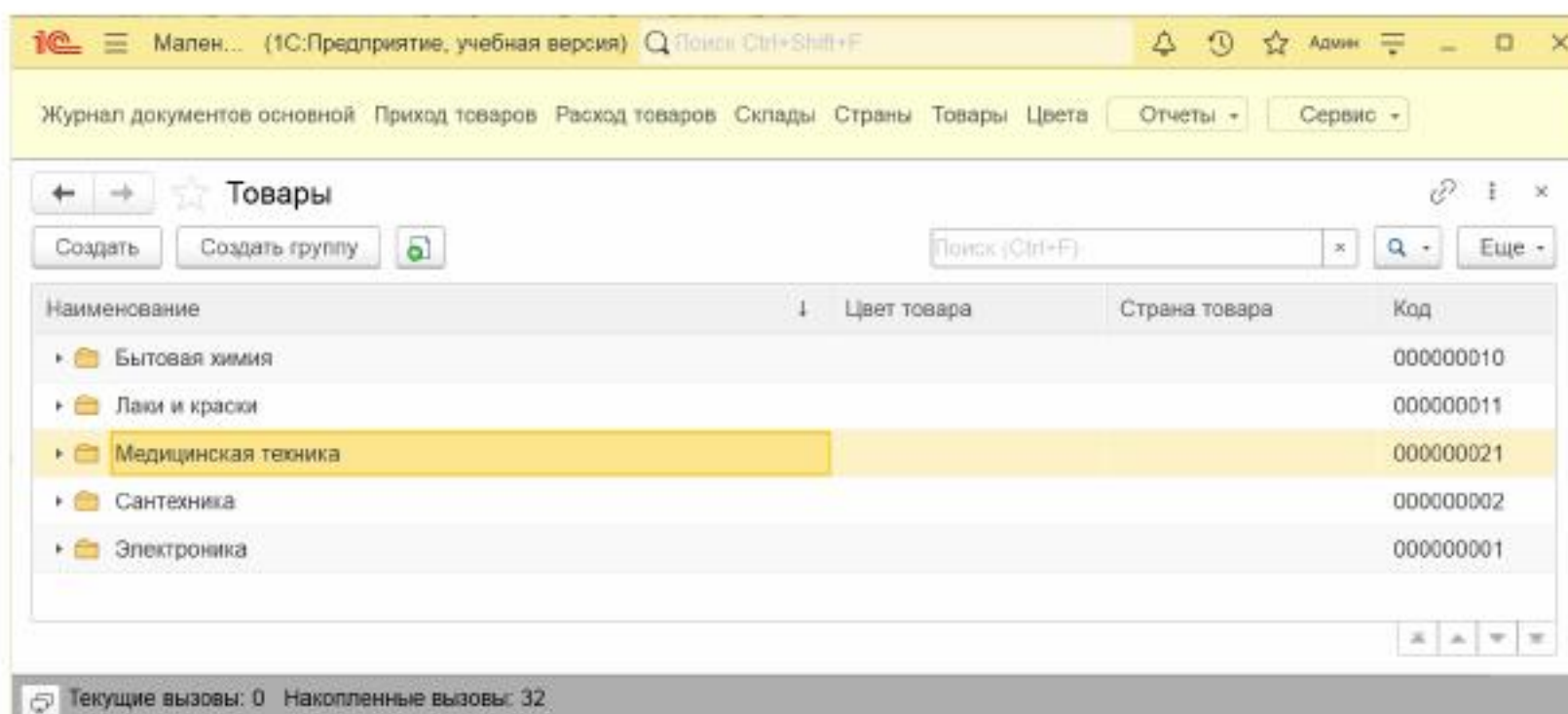


Рис. 11. Результат работы обработки «Создать группу»

Создание команды «Выгрузить в текстовый файл»

Выполним реализацию обработки, которая будет позволять выгружать товары в текстовый файл из выбранной пользователем группы товаров. В верхней правой части окна редактора форм перейдем на вкладку «Реквизиты» и добавим новый реквизит «ВыбрКатегория». В его свойствах, для свойства «Тип» выберем СправочникСсылка.Товары (см. рис. 12).

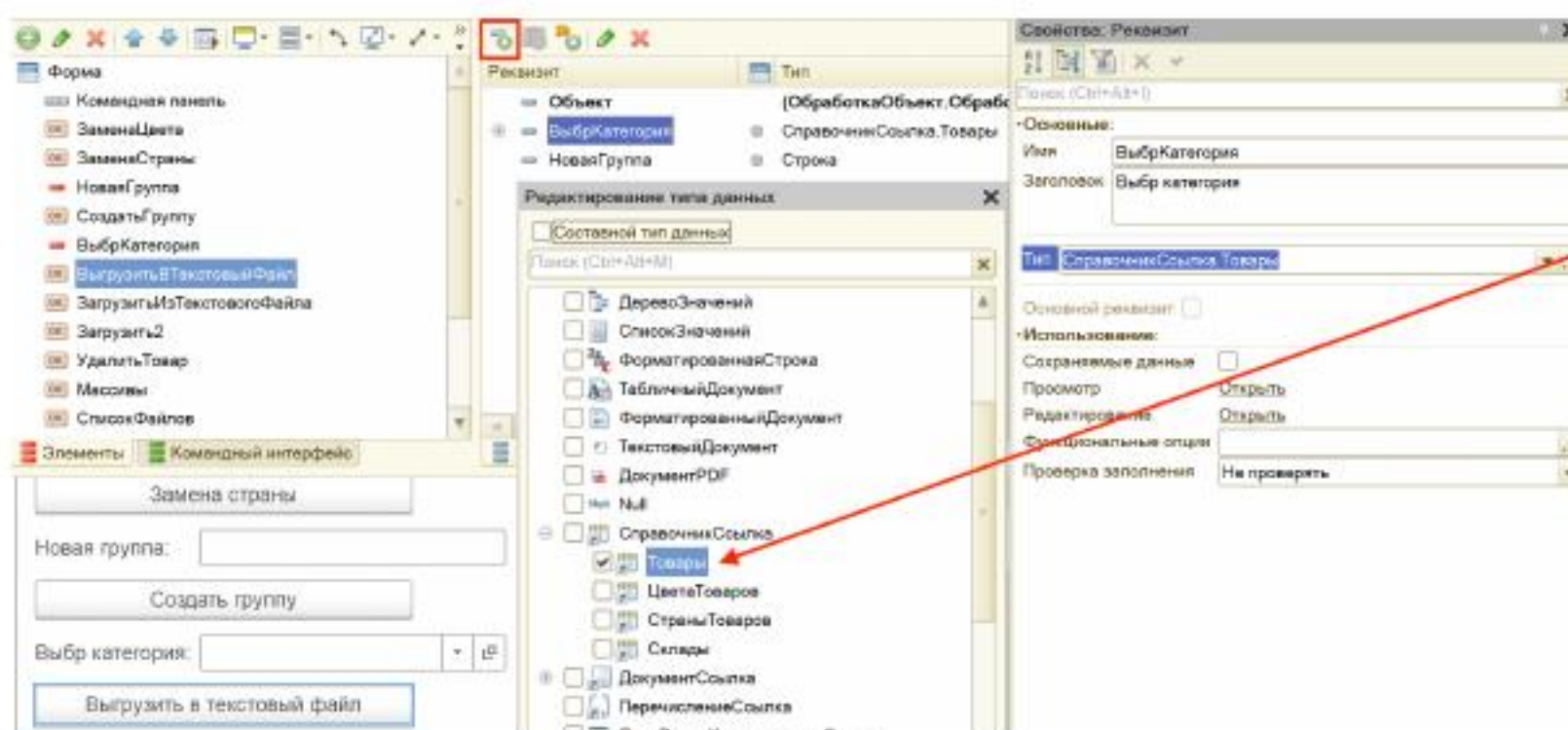


Рис. 12. Добавление реквизита «ВыбрКатегория»

Далее, перетащим его в левую часть к элементам формы, после чего этот компонент отобразится на нашей форме. Чтобы у пользователя была возможность не вводить наименование группы, а выбирать нужную категорию товаров из списка групп, не давая при этом выбрать элемент, необходимо в свойствах этого компонента найти свойство «ВыборГруппИЭлементов» и выставить для него значение «Групп» (см. рис. 13).

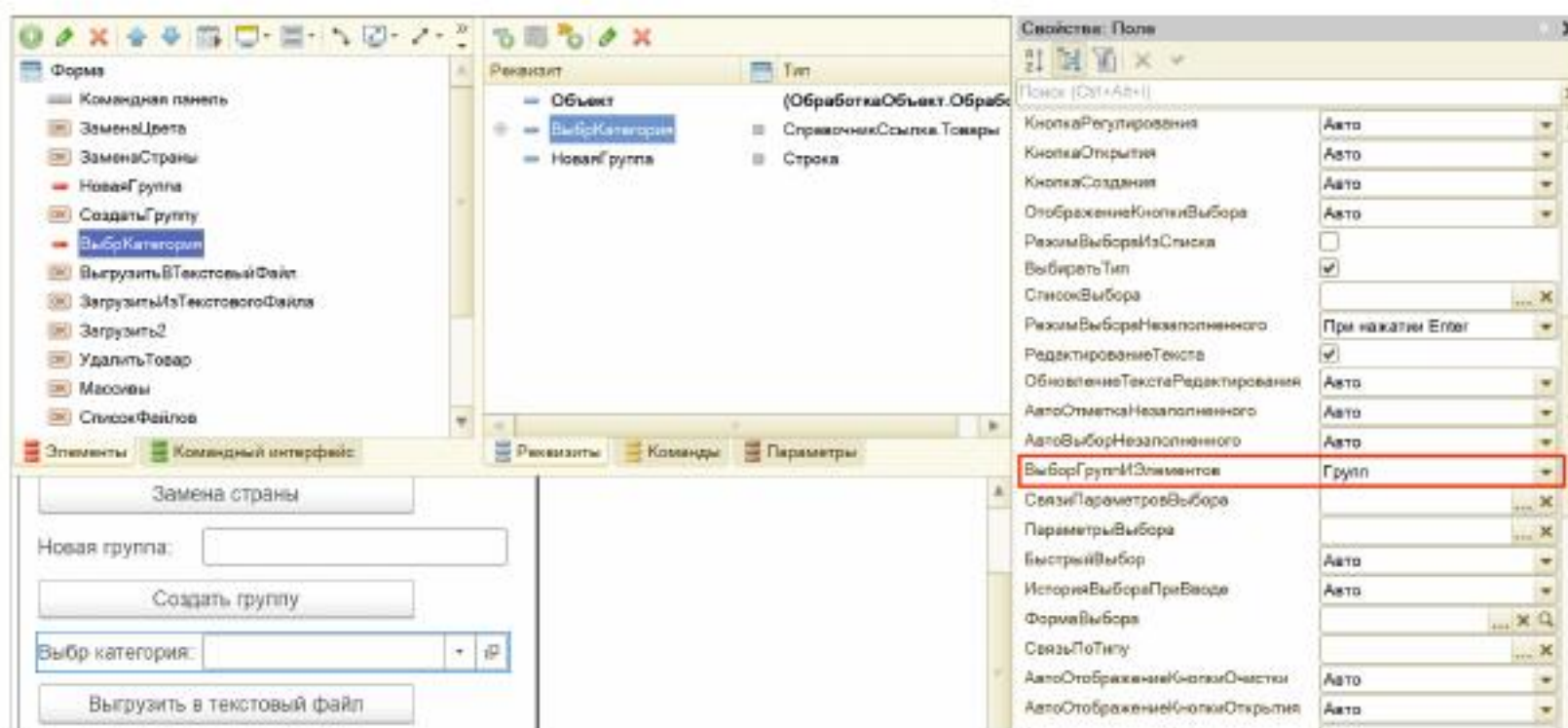


Рис. 13. Добавление на форму поля ввода «ВыбрКатегория»

Теперь, по аналогии с предыдущими примерами, добавим команду «ВыгрузитьВТекстовыйФайл» и запрограммируем её следующим образом.

&НаСервереБезКонтекста

Процедура ВыгрузитьВТекстовыйФайлНаСервере(ВыбКат)

Текст = Новый ТекстовыйДокумент;

НужнаяГруппаТоваров

=

Справочники.Товары.НайтиПоНаименованию(СокрЛП(ВыбКат));

ВыборкаТоваров

=

Справочники.Товары.Выбрать(НужнаяГруппаТоваров);

Пока ВыборкаТоваров.Следующий() Цикл

Если ВыборкаТоваров.ЭтоГруппа Тогда

Продолжить;

КонецЕсли;

КодТовара = Строка(ВыборкаТоваров.Код);

НаименованиеТовара =
Строка(ВыборкаТоваров.Наименование);

Текст.ДобавитьСтроку(КодТовара + ";" +
НаименованиеТовара);
КонецЦикла;
Текст.Записать("C:\Users\alexeysemenov\Desktop\2345.txt");

КонецПроцедуры

&НаКлиенте

Процедура ВыгрузитьВТекстовыйФайл(Команда)
ВыгрузитьВТекстовыйФайлНаСервере(ВыбрКатегория);
КонецПроцедуры

В этом программном коде реализована следующая логика. Вначале, при нажатии на кнопку «Выгрузить в текстовый файл», на клиенте вызывается процедура *ВыгрузитьВТекстовыйФайл(Команда)*, которая в свою очередь будет запускать на сервере процедуру *ВыгрузитьВТекстовыйФайлНаСервере(ВыбрКатегория)*, которая в качестве параметра принимает выбранную пользователем группу товаров. Логика работы процедуры, выполняющейся на сервере, следующая.

Значение аргумента *ВыбрКатегория* на сервере мы принимаем в параметре *ВыбКат*. Вначале создаем значение «ТекстовыйДокумент» и записываем его в переменную *Текст*. Таким образом мы создали объект для работы с текстовым файлом. Далее, в переменную *ВыборкаТоваров* записываем только те товары, которые принадлежат выбранной пользователем категории. Затем, прогоняем в цикле все товары из нашей выборки и если текущий элемент не является группой, получаем значения необходимых реквизитов и формируем строку для добавления в текстовый файл, используя в качестве разделителя точку с запятой: *Текст.ДобавитьСтроку(КодТовара + ";" + НаименованиеТовара)*. После окончания цикла осуществляем запись в текстовый файл с помощью метода *Записать()*.

В завершении, остается проверить код на ошибки, создать кнопку для этой команды и проверить функционирование обработки в пользовательском режиме 1С (см. рис. 14, 15, 16).

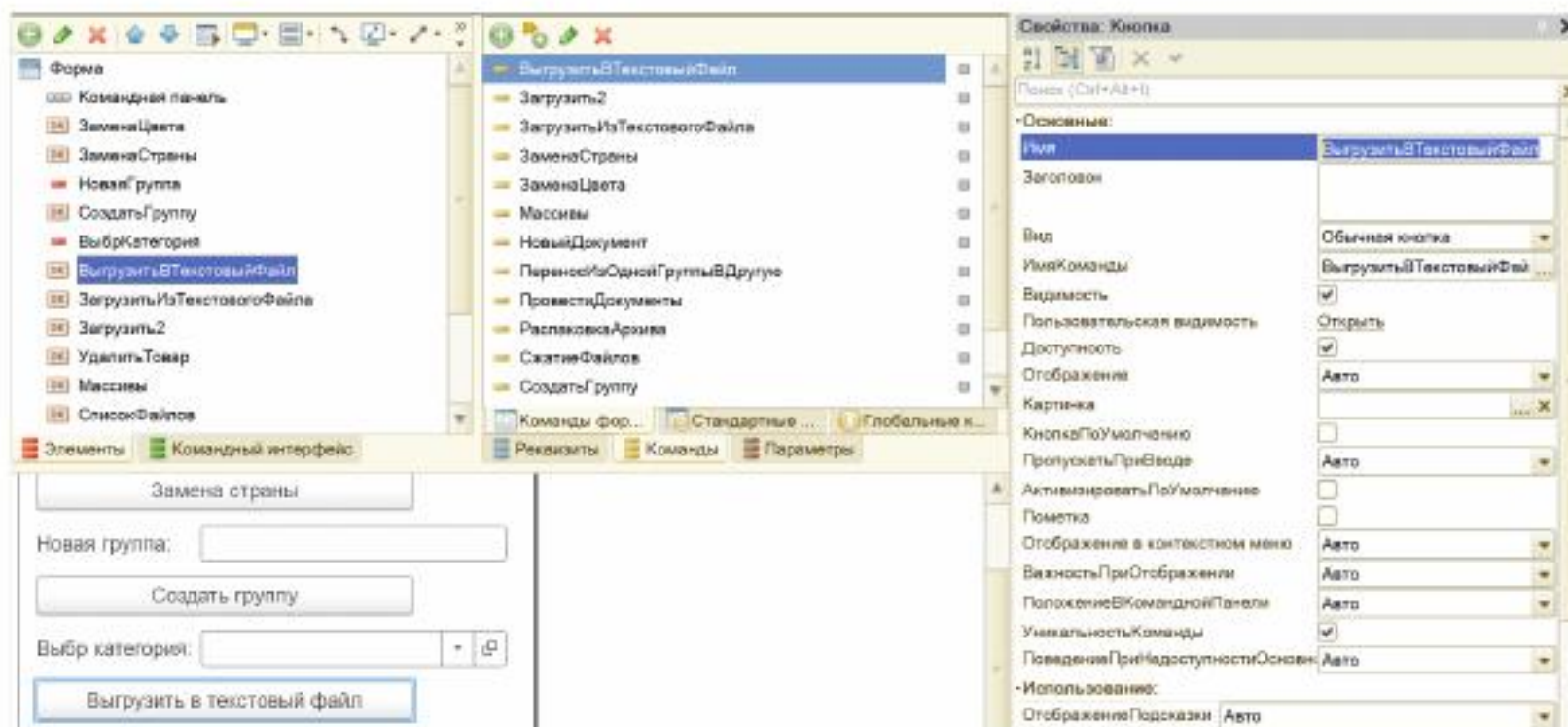


Рис. 14. Создание кнопки «Выгрузить в текстовый файл» на форме

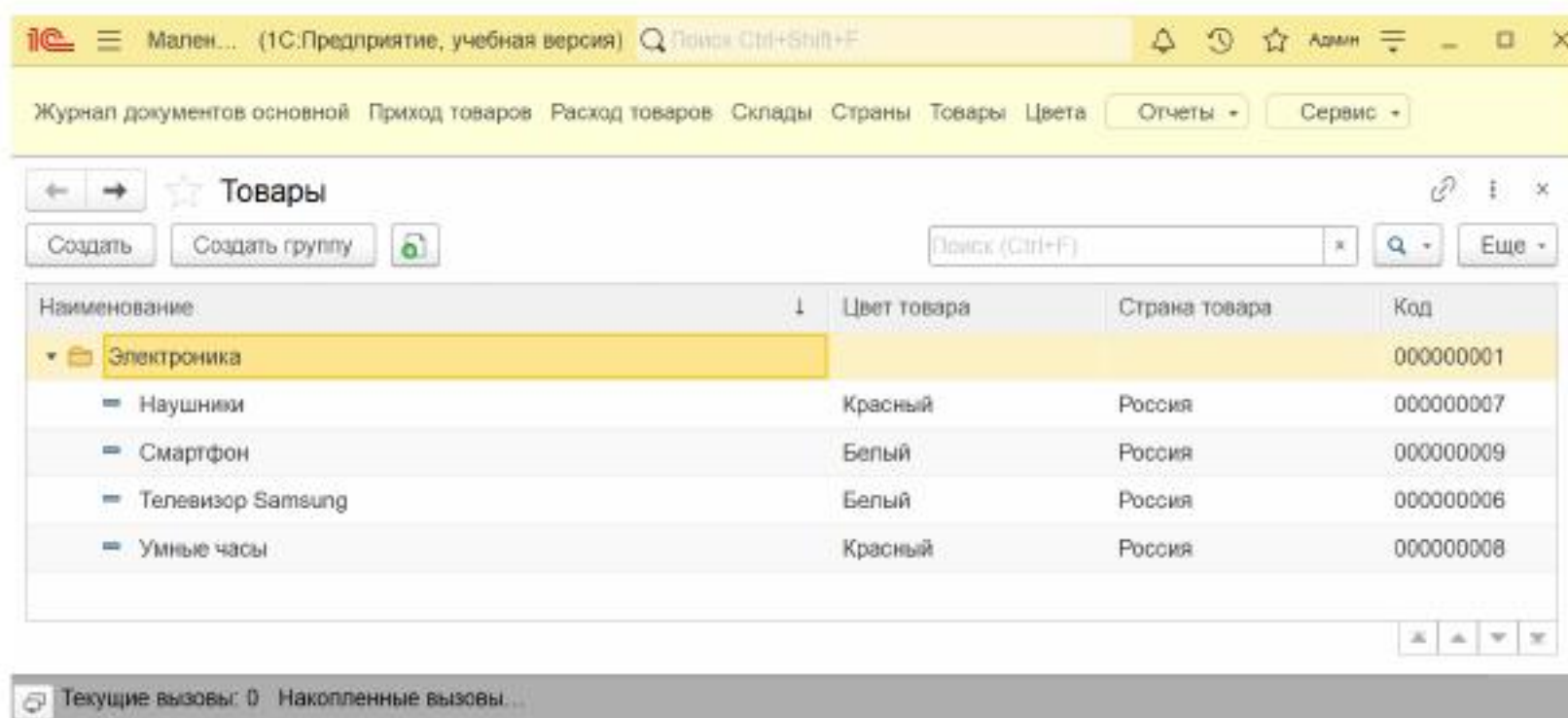


Рис. 15. Список товаров в папке Электроника перед выгрузкой

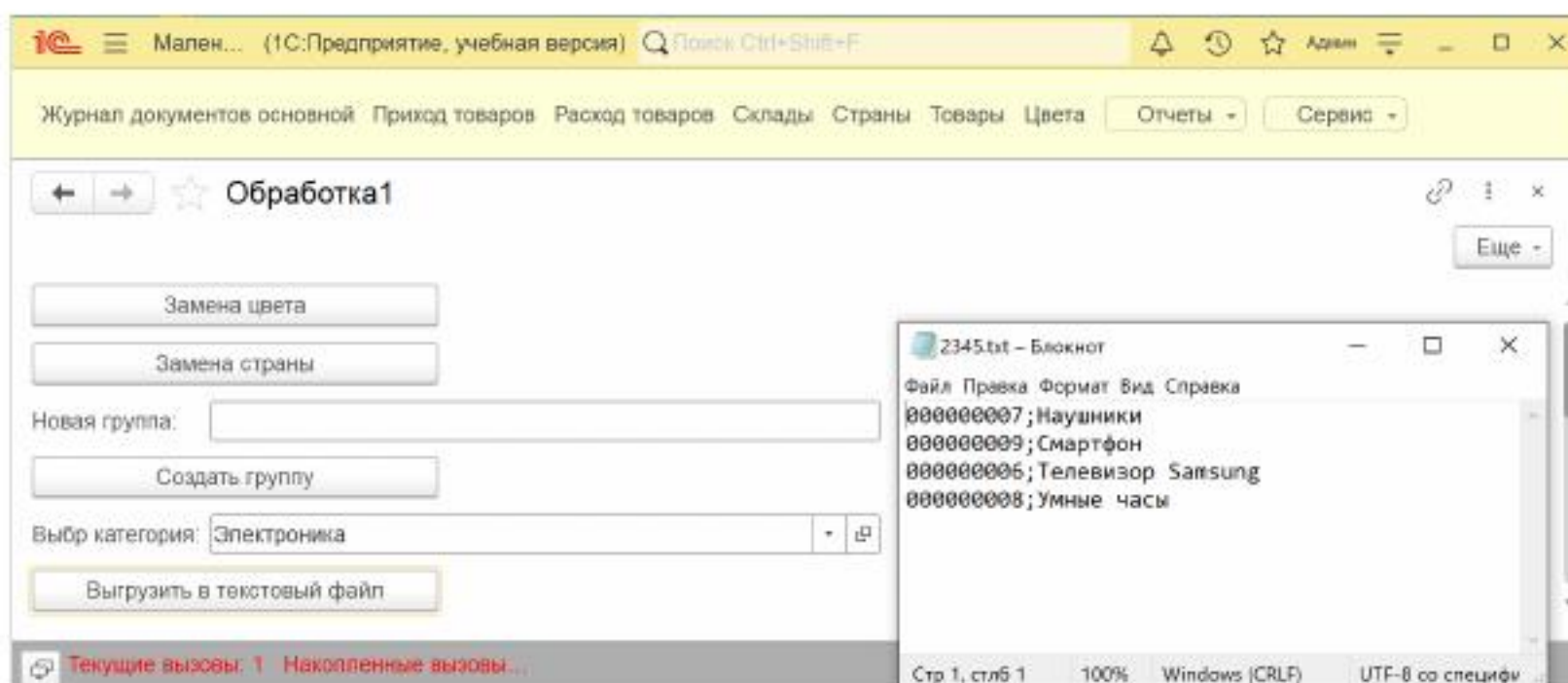


Рис. 16. Результат работы обработки «Выгрузить в текстовый файл»

Создание команды «Загрузить из текстового файла»

Реализуем возможность загружать в указанную категорию товары из текстового файла. Предположим, что в качестве текстового файла мы будем использовать файл, полученный с помощью предыдущей обработки. Также воспользуемся уже созданным ранее полем ввода «ВыборКатегория». Аналогично предыдущим примерам создадим команду «ЗагрузитьИзТекстовогоФайла» (см. рис. 17).

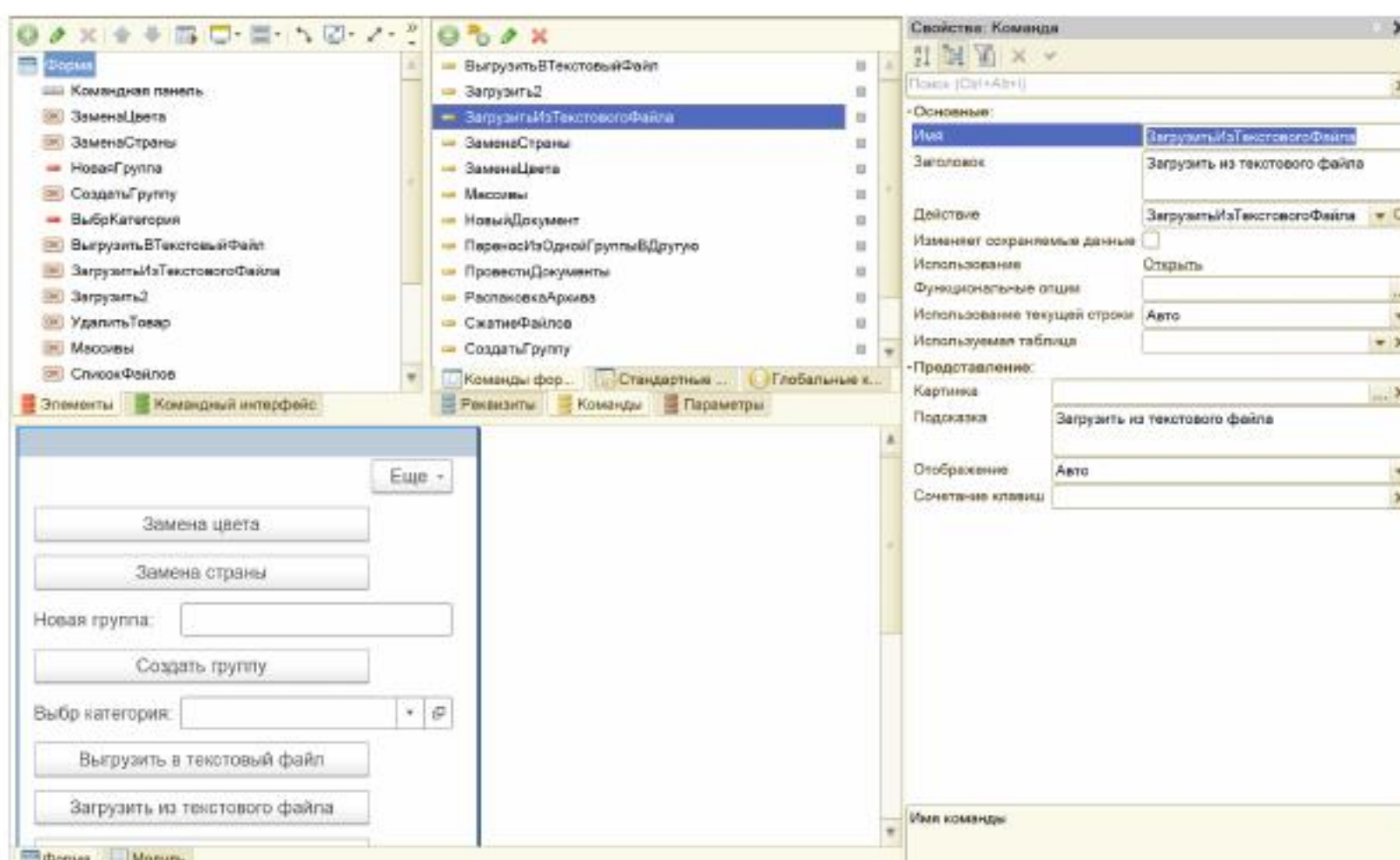


Рис. 17. Добавление команды «ЗагрузитьИзТекстовогоФайла»

Приведём программный код данной команды.

&НаСервереБезКонтекста

Процедура ЗагрузитьИзТекстовогоФайлаНаСервере(ВыбКат)

Текст = Новый ТекстовыйДокумент;

Текст.Прочитать("C:\Users\alexeysemenov\Desktop\2345.txt");

ФлагЗадвойки=0;

Для НомерСтроки=1 По Текст.КоличествоСтрок() Цикл

ТекущаяСтрока = Текст.ПолучитьСтроку(НомерСтроки);

ПозицияРазделителя = СтрНайти(ТекущаяСтрока, ";");

КодТовара = Сред(ТекущаяСтрока, 1, ПозицияРазделителя-1);

НаимТовара = Сред(ТекущаяСтрока, ПозицияРазделителя+1);

НужнаяГруппаТоваров

=

Справочники.Товары.НайтиПоНаименованию(СокрЛП(ВыбКат));

ВыборкаТоваров

=

Справочники.Товары.Выбрать(НужнаяГруппаТоваров);

Пока ВыборкаТоваров.Следующий() Цикл

Если ВыборкаТоваров.ЭтоГруппа Тогда

Продолжить;

КонецЕсли;

ТекущийТовар

=

ВыборкаТоваров.Ссылка.ПолучитьОбъект();

Если СокрЛП(ТекущийТовар.Наименование)

=

СокрЛП(НаимТовара) Тогда

Сообщить("Такой товар уже есть.");

ФлагЗадвойки=1;

Прервать;

КонецЕсли;

КонецЦикла;

Если ФлагЗадвойки=0 Тогда

НовыйТовар = Справочники.Товары.СоздатьЭлемент();

НовыйТовар.Наименование = НаимТовара;

НовыйТовар.Родитель = ВыбКат;

НовыйТовар.Записать();


```

Сообщить("Товар " + НаимТовара + " загружен в
категорию " + ВыбКат);
КонецЕсли;
ФлагЗадвойки=0;
КонецЦикла;
КонецПроцедуры

```

&НаКлиенте

Процедура ЗагрузитьИзТекстовогоФайла(Команда)

ЗагрузитьИзТекстовогоФайлаНаСервере(ВыбрКатегория);

КонецПроцедуры

Логика представленного программного кода заключается в следующем. На клиенте срабатывает процедура, которая запускает процедуру на сервере и передает ей выбранную на форме категорию товаров. В серверной процедуре, мы вначале создаем значение «ТекстовыйДокумент» и записываем его в переменную Текст. Таким образом мы создали объект для работы с текстовым файлом. Далее, считываем текстовый документ из файла. В цикле осуществляем построчный перебор и ориентируясь на разделитель «;» выделяем из строки код товара и наименование товара. Делаем выборку товаров из указанной пользователем категории и начинаем сравнивать товар, считанный из файла, с выборкой товаров. Если в ней уже имеется точно такой же товар, выводим сообщение об этом и меняем значение флага, иначе создаем в этой категории новый элемент, заполняем его реквизиты считанными из файла значениями и записываем этот новый элемент в базу.

Проверим код на ошибки, создадим кнопку для этой команды (см. рис. 18).

Проверим работу обработки в пользовательском режиме 1С. Создадим для этого тестовую категорию, например: Электроника 2. Затем запустим нашу обработку, выберем данную категорию и удостоверимся, что импорт данных из файла произошел успешно (см. рис. 19).

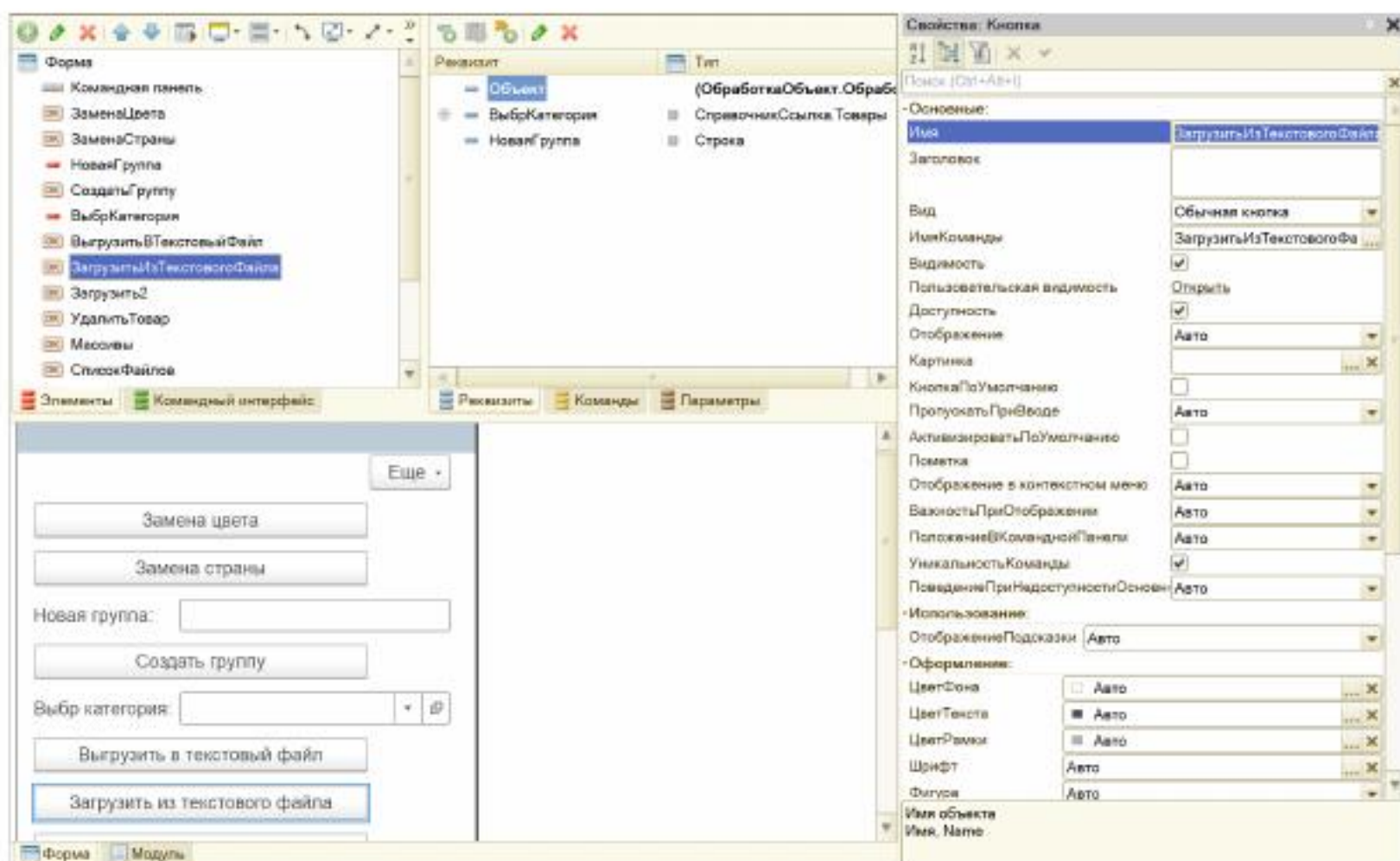
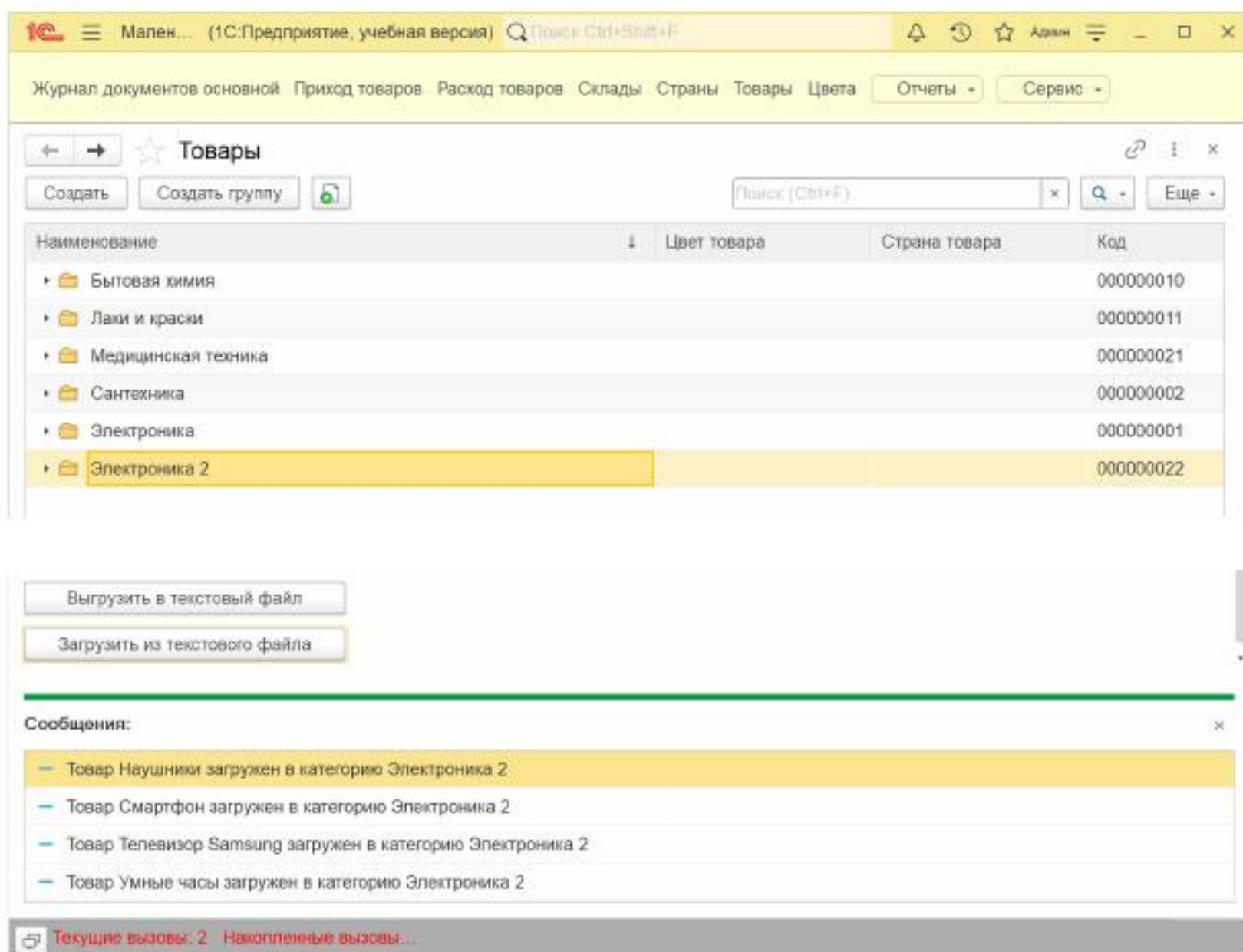


Рис. 18. Создание кнопки «Загрузить из текстового файла»



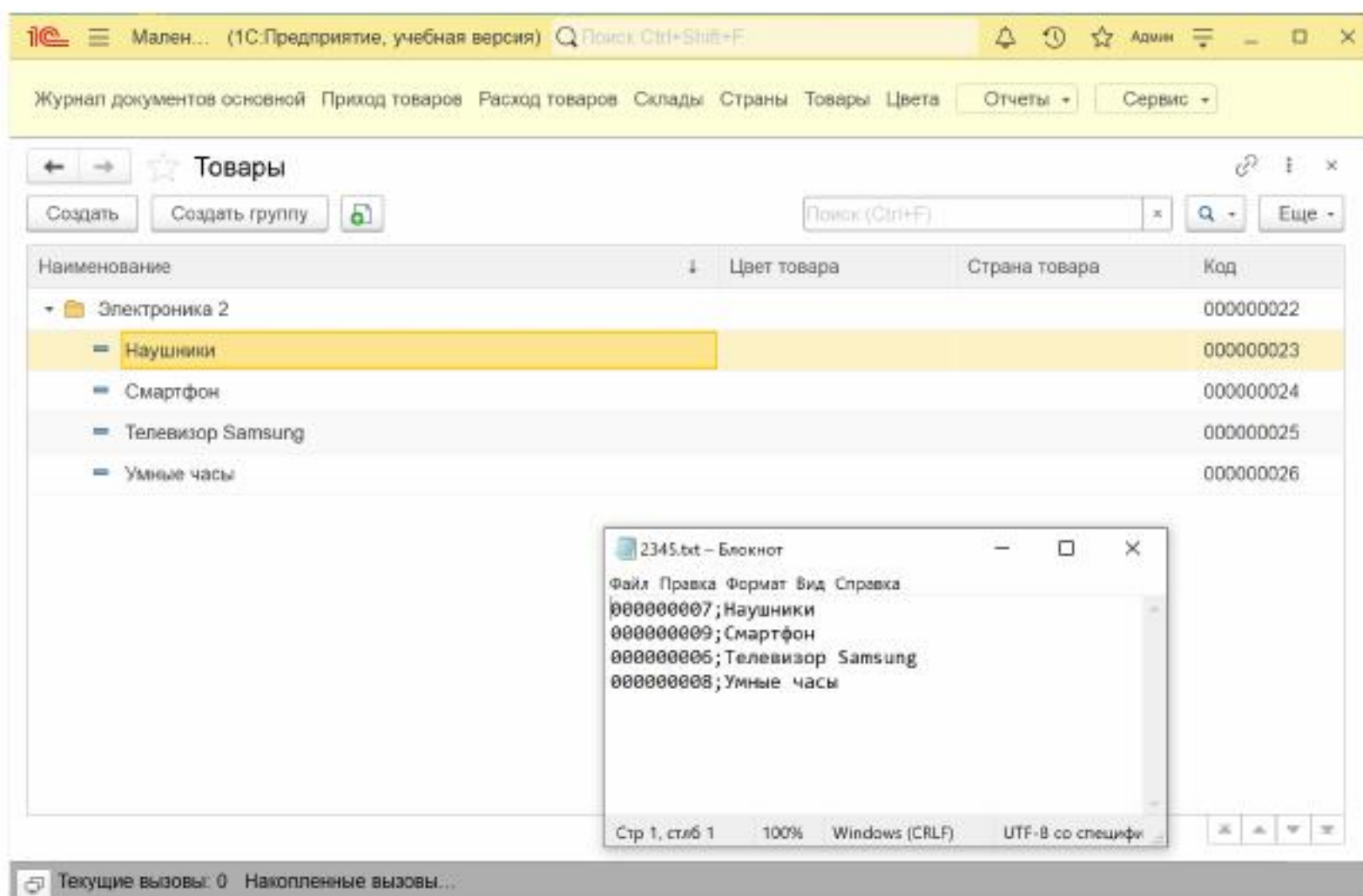


Рис. 19. Результат работы обработки «Загрузить из текстового файла»

Как видно из скриншота, такие свойства, как цвет товара и страна не заполнены. Это верно, так как мы их не выгружали в файл и соответственно не считывали.

Считывание текста из текстового документа можно осуществлять ещё другим способом. Создадим команду «Загрузить2» со следующим программным кодом.

&НаСервереБезКонтекста

Процедура Загрузить2НаСервере()

Текст = *Новый*

ЧтениеТекста("C:\Users\alexeysemenov\Desktop\2345.txt",

КодировкаТекста.ANSI);

ТекущаяСтрока = Текст.ПрочитатьСтроку();

Пока ТекущаяСтрока <> Неопределено Цикл

ПозицияРазделителя = СтрНайти(ТекущаяСтрока, ";");

КодТовара = Сред(ТекущаяСтрока, 1, ПозицияРазделителя-1);

НаимТовара = Сред(ТекущаяСтрока, ПозицияРазделителя+1);

Сообщить(КодТовара + " " + НаимТовара);

ТекущаяСтрока = Текст.ПрочитатьСтроку();

КонецЦикла;

КонецПроцедуры

&НаКлиенте

Процедура Загрузить2(Команда)

Загрузить2НаСервере();

КонецПроцедуры

В этом примере мы использовали для считывания файла методы *ЧтениеТекста()* для последовательного чтения текстовых файлов большой длины и *ПрочитатьСтроку()*. Этот вариант хорошо подходит для больших файлов.

Создание команды «Удалить товар»

Рассмотрим пример реализации обработки, позволяющей удалить товар, если он не задействован ни в каких документах, то есть если не найдены ссылки на этот элемент. В качестве справочной информации стоит отметить, что если удалить какой-либо элемент справочника без ссылочного контроля, то это может привести к нарушению точки актуальности и границы последовательности. Например, пусть имеется некий товар, который фигурирует в приходных и расходных документах. Если этот товар удалить без всяких проверок, то в документах возникнут ссылки на несуществующий элемент, что приведет к нарушению целостности данных. А вот если этот товар не был задействован ни в каких документах, то никаких проблем с целостностью данных не возникнет после его удаления. Создадим команду «Удалить товар» (см. рис. 20).

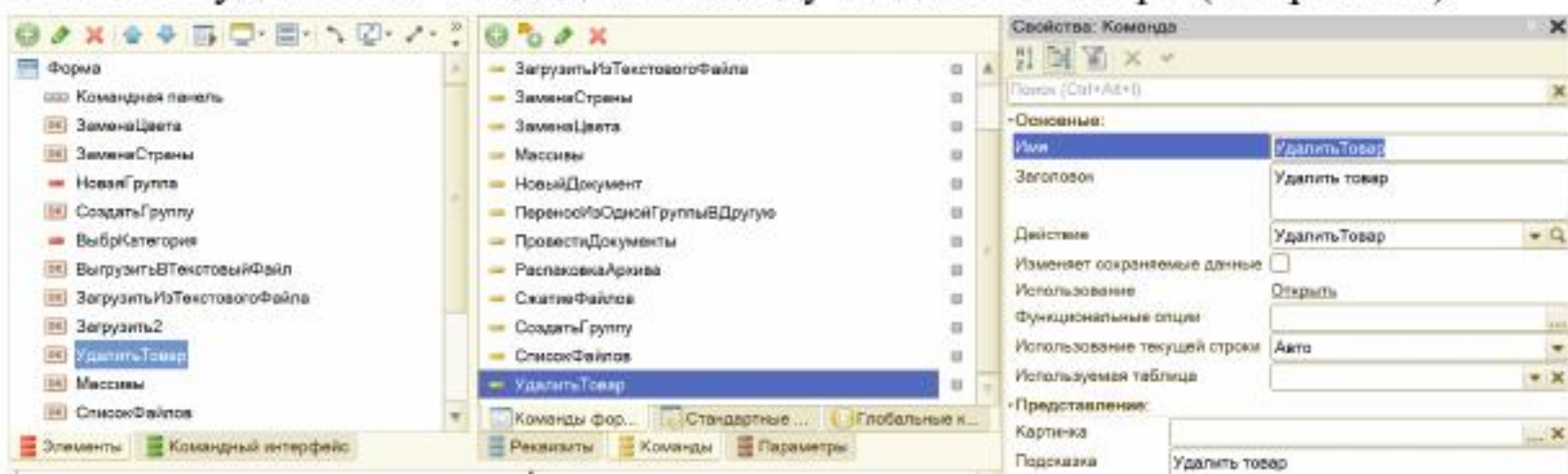


Рис. 20. Добавление команды «УдалитьТовар»

Запрограммируем команду следующим образом.

&НаСервереБезКонтекста

Процедура УдалитьТоварНаСервере()

Товар = Справочники.Товары;

УдаляемыйТоварСсылка = Товар.НайтиПоКоду("000000007");

МассивСсылок = Новый Массив;

МассивСсылок.Добавить(УдаляемыйТоварСсылка);

НайденныеСсылки = НайтиПоСсылкам(МассивСсылок);

УдаляемыйТовар = УдаляемыйТоварСсылка.ПолучитьОбъект();

Если НайденныеСсылки.Количество() > 0 Тогда

Сообщить("Найдены ссылки на элемент! Удалить нельзя!");

УдаляемыйТовар.УстановитьПометкуУдаления(Истина);

Иначе

Сообщить("Товар " +

СокрЛП(УдаляемыйТоварСсылка.Наименование) + " удален!");

УдаляемыйТовар.Удалить();

КонецЕсли;

КонецПроцедуры

&НаКлиенте

Процедура УдалитьТовар(Команда)

УдалитьТоварНаСервере();

КонецПроцедуры

В данном коде происходит следующее. Предположим, нам необходимо удалить товар с кодом 000000007. Вначале мы находим этот товар в справочнике товаров по коду и получаем ссылку на него. Далее мы создаем массив для хранения ссылок на товар и с помощью метода *НайтиПоСсылкам* осуществляем поиск ссылок на объект. Затем мы получаем сам объект, чтобы иметь возможность удалить его, так как со ссылкой на объект мы не сможем выполнить его удаление. Производим проверку на количество ссылок. Если ссылки есть, то выводим сообщение

о невозможности удаления этого товара и ставим просто пометку на удаление, иначе спокойно удаляем с выводом сообщения об удалении товара.

Создадим кнопку и проверим обработку в пользовательском режиме 1С (см. рис. 21, 22).

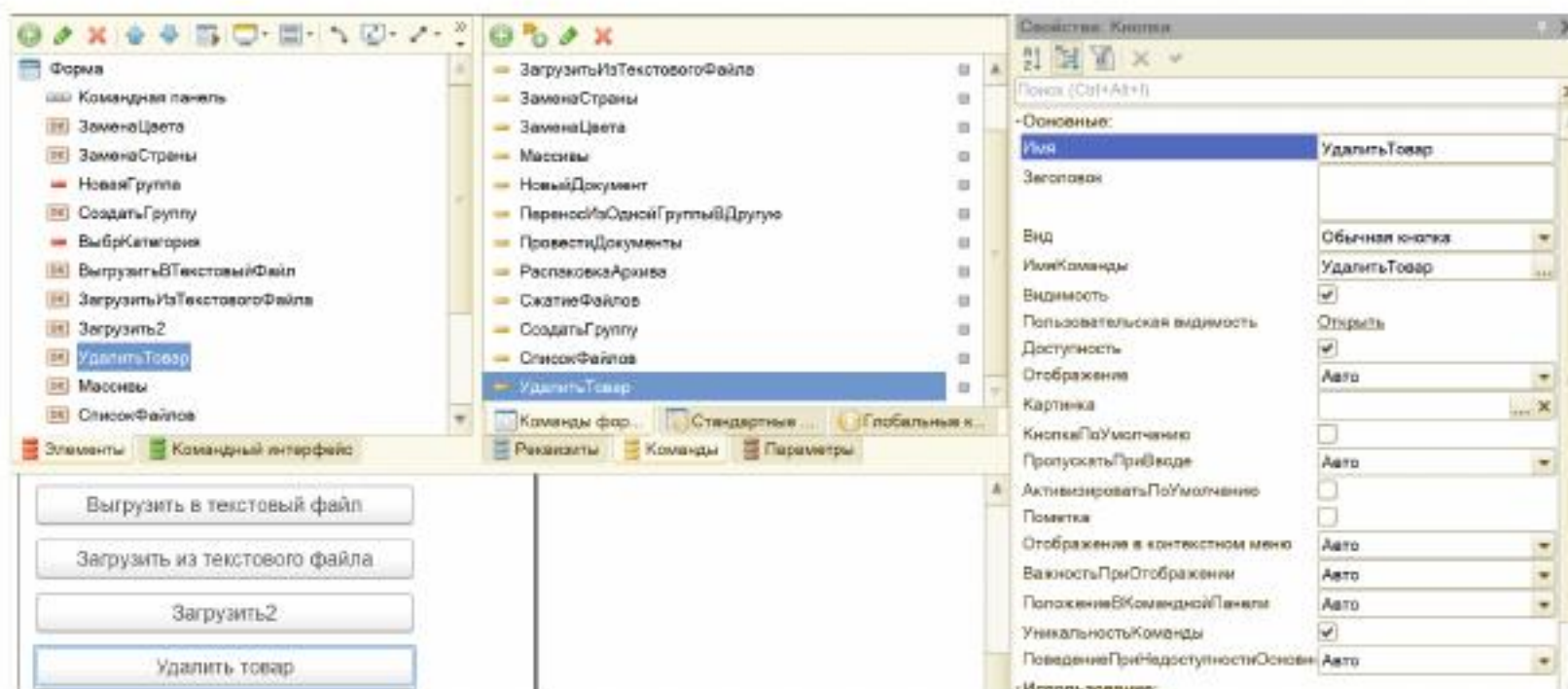
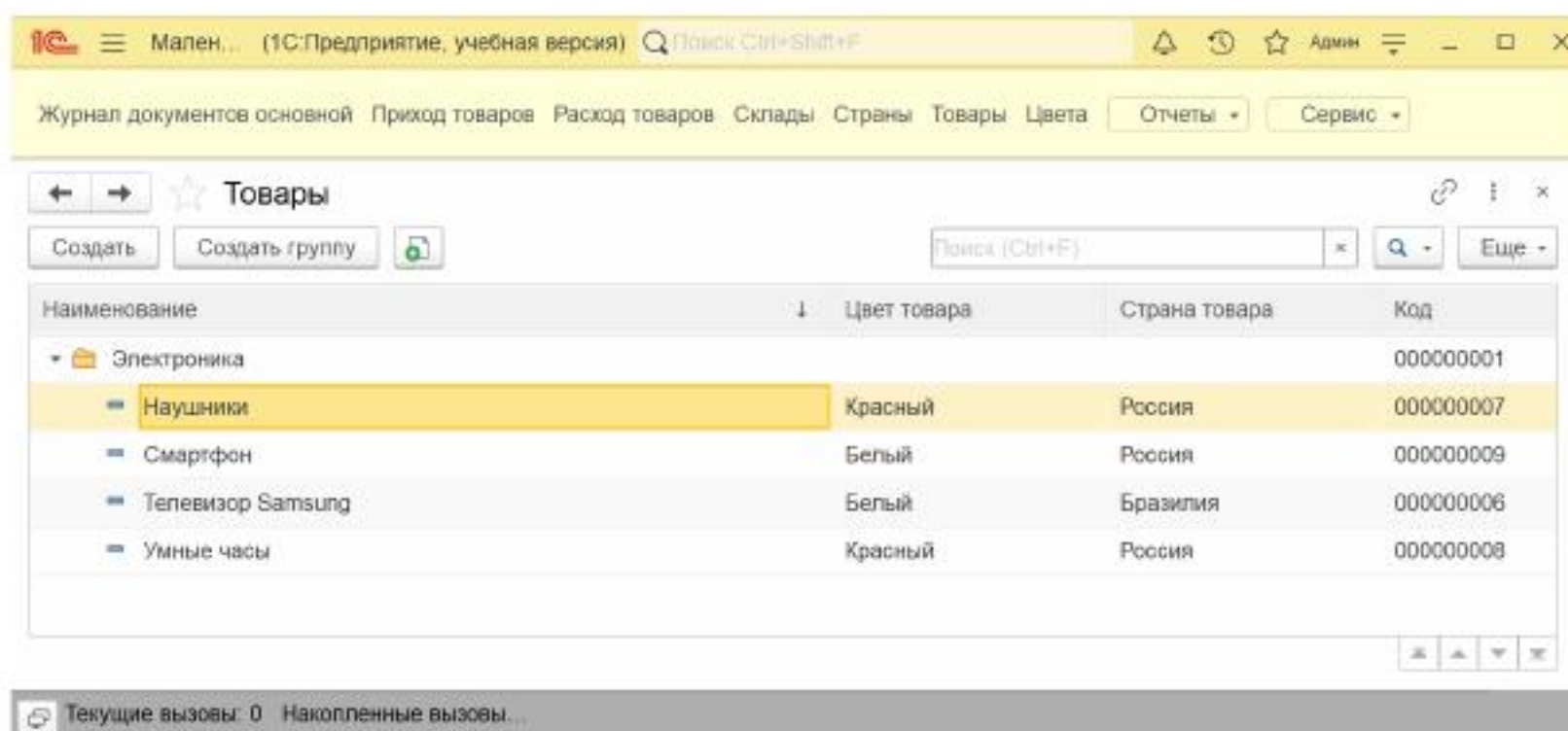


Рис. 21. Создание кнопки «Удалить товар»

Товар с кодом 000000007 – наушники. Они фигурируют в нескольких приходных и расходных документах. Таким образом, обработка не должна удалить этот элемент.



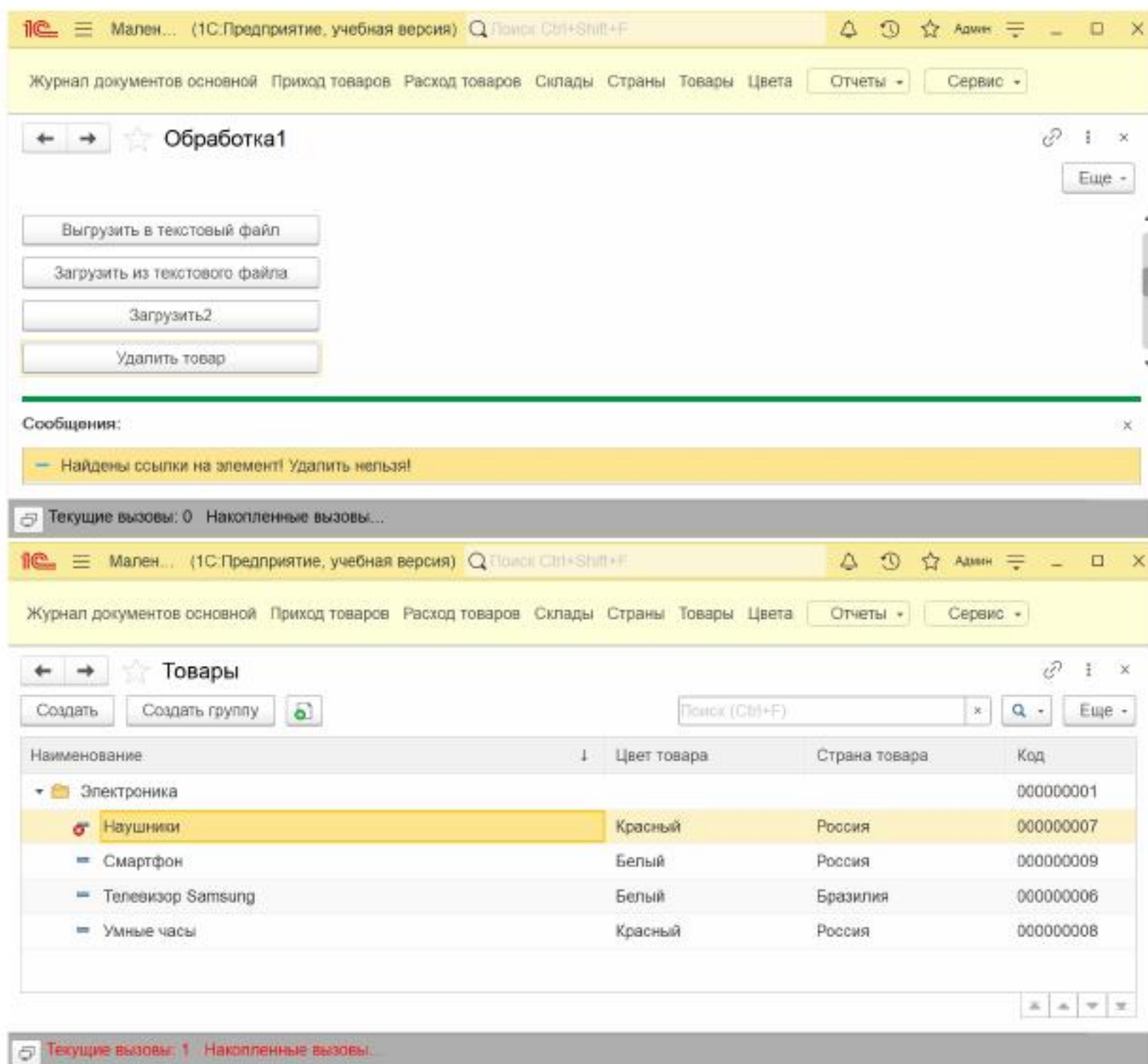


Рис. 22. Результат работы обработки «Удалить товар»

А вот если мы захотим удалить товар с кодом 0000000008 – умные часы, то обработка выполнит удаление этого элемента, так как он ни в каких документах не фигурирует и соответственно ссылок на него нет (см. рис. 23).



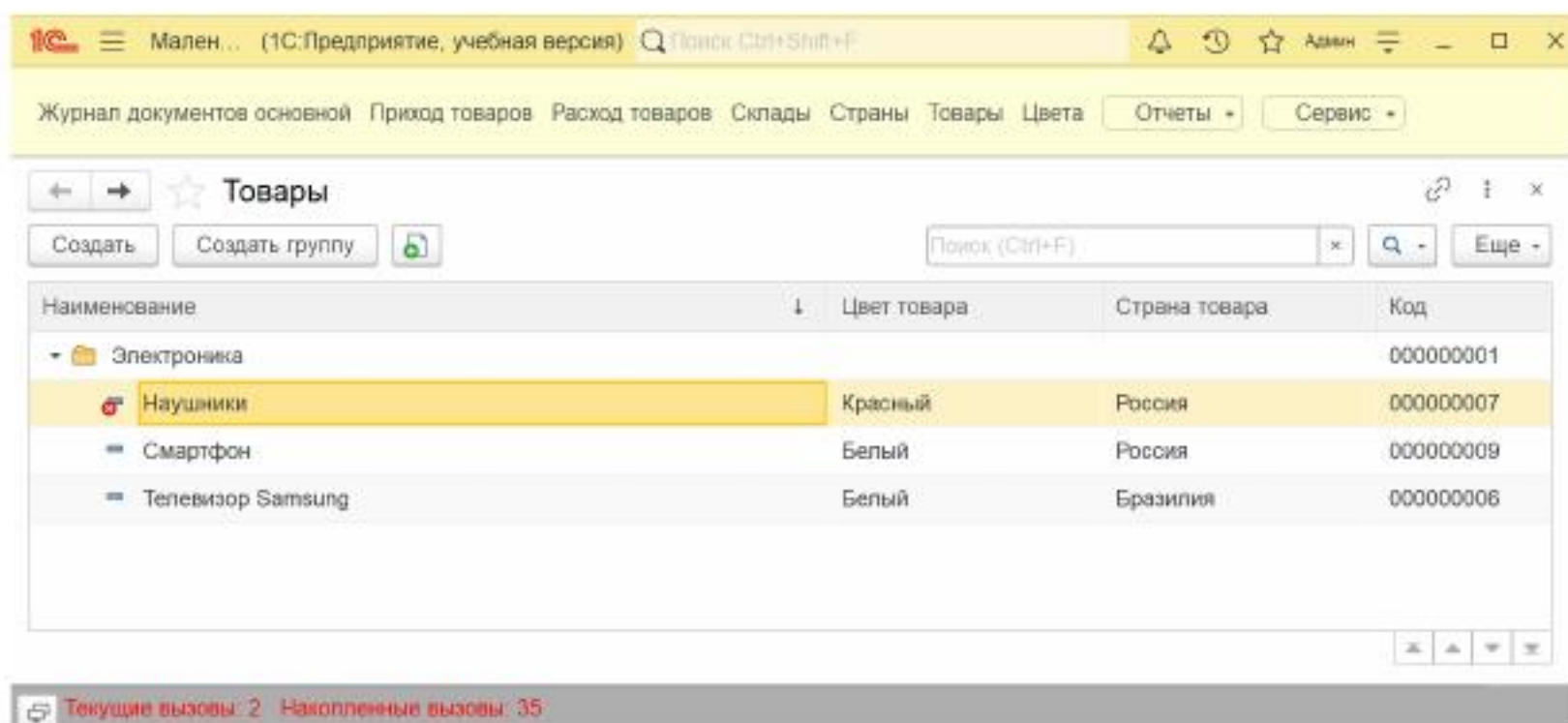


Рис. 23. Результат работы обработки «Удалить товар»

Создание команды «Список файлов»

Рассмотрим реализацию команды, позволяющей вывести список файлов с определенным расширением по заданному пути. Вначале, как обычно, добавим новую команду (см. рис. 24) и запрограммируем её следующим образом.

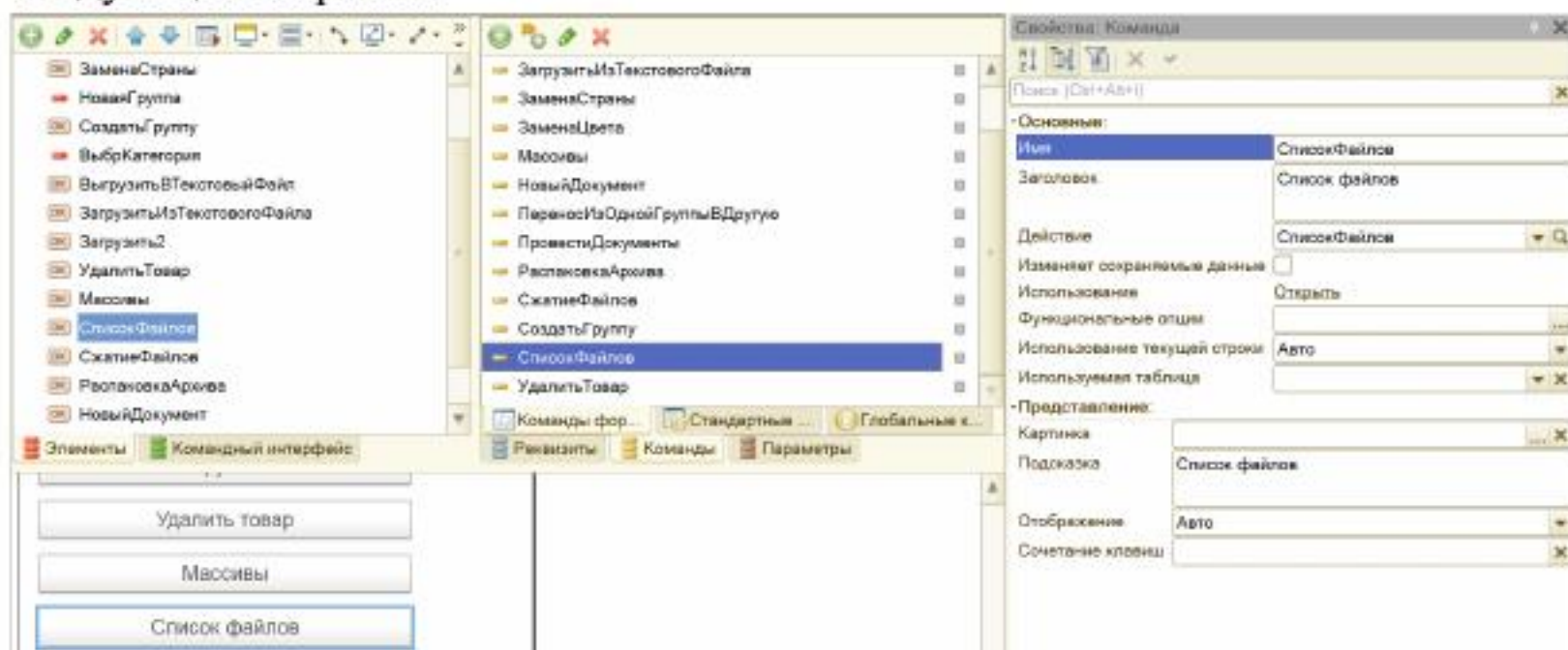


Рис. 24. Добавление команды «СписокФайлов»

&НаСервереБезКонтекста

Процедура СписокФайловНаСервере()

МассивФайлов

=

НайтиФайлы("C:\Users\alexeysemenov\Desktop\Можно удалить\",".txt");*

Файл = Новый Файл("");

Для каждого Файл Из МассивФайлов Цикл

Сообщить(Файл.ПолноеИмя);
КонецЦикла;
КонецПроцедуры

&НаКлиенте
Процедура СписокФайлов(Команда)
СписокФайловНаСервере();
КонецПроцедуры

Здесь используется функция *НайтиФайлы*, которая осуществляет поиск файлов и каталогов по заданной маске, расположенных в заданном каталоге, и возвращает массив из значений типа *Файл*, содержащий найденные файлы. В нашем примере этот массив мы записываем в переменную *МассивФайлов*. Функция *НайтиФайлы* довольно часто используется, поэтому рассмотрим её более подробно.

Синтаксис:

НайтиФайлы(<Путь>, <Маска>, <ИскатьВПодкаталогах>)

Параметры:

<Путь> (обязательный)

Тип: Строка. Путь к каталогу, в котором производится поиск. Каталог может быть как на локальной сети, так и на ftp-сервере. При использовании этой схемы в адресах необходимо указывать прямые слэши '/', а не обратные '\'. Допускается указание полного имени файла (путь + имя).

<Маска> (необязательный)

Тип: Строка. Маска выбора файлов. Если параметр задан, то первый параметр воспринимается системой как путь к каталогу, в котором требуется найти файлы, удовлетворяющие маске выбора. Про комбинации специальных символов для различных операционных систем можно получить информацию в справочной системе 1С.

<ИскатьВПодкаталогах> (необязательный)

Тип: Булево. Истина - будет произведен обход вложенных в параметр *<Путь>* каталогов и поиск в них. Параметр игнорируется, если не задан параметр *<Маска>*, т.к. предполагается, что ищется один файл. Значение по умолчанию: Ложь.

Далее, мы создаем объект типа Файл, который используется для доступа к параметрам файлов или каталогов. Необходимо отметить, что данный объект не предназначен для чтения и записи из файлов. Затем, в цикле перебираем полученный нами массив и выводим в сообщении полное имя файла. Естественно, вместо вывода сообщения могут быть запрограммированы другие операции с файлами.

Добавим кнопку и проверим обработку в пользовательском режиме 1С (см. рис. 25). Действительно, обработка находит только файлы с расширением txt в указанной папке и выводит их перечень в сообщении.

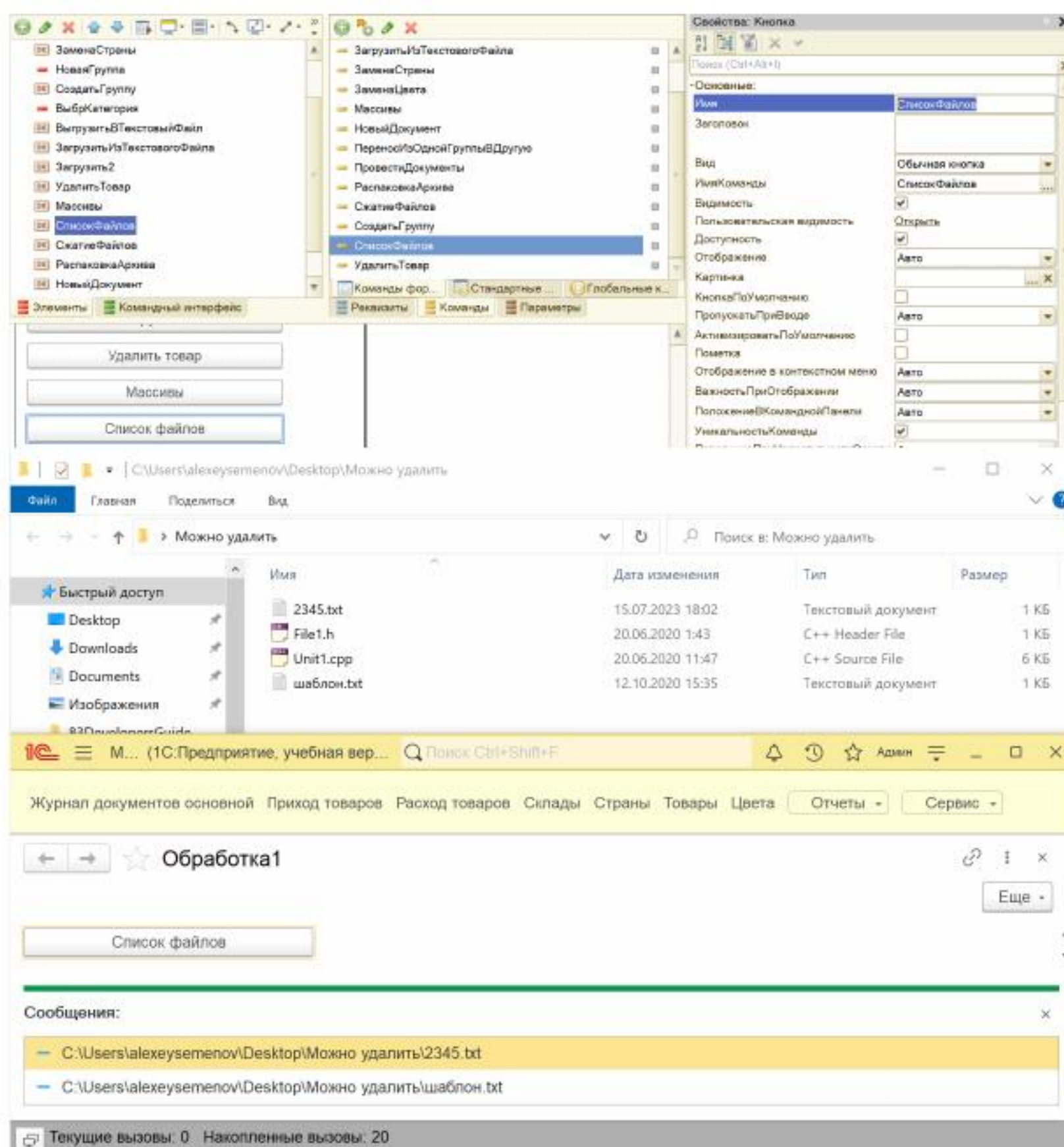


Рис. 25. Результат работы обработки «Список файлов»

Создание команды «Сжатие файлов»

Рассмотрим реализацию команды, позволяющей архивировать группу файлов с определенным расширением в заданной папке. Вначале, добавим новую команду (см. рис. 26).

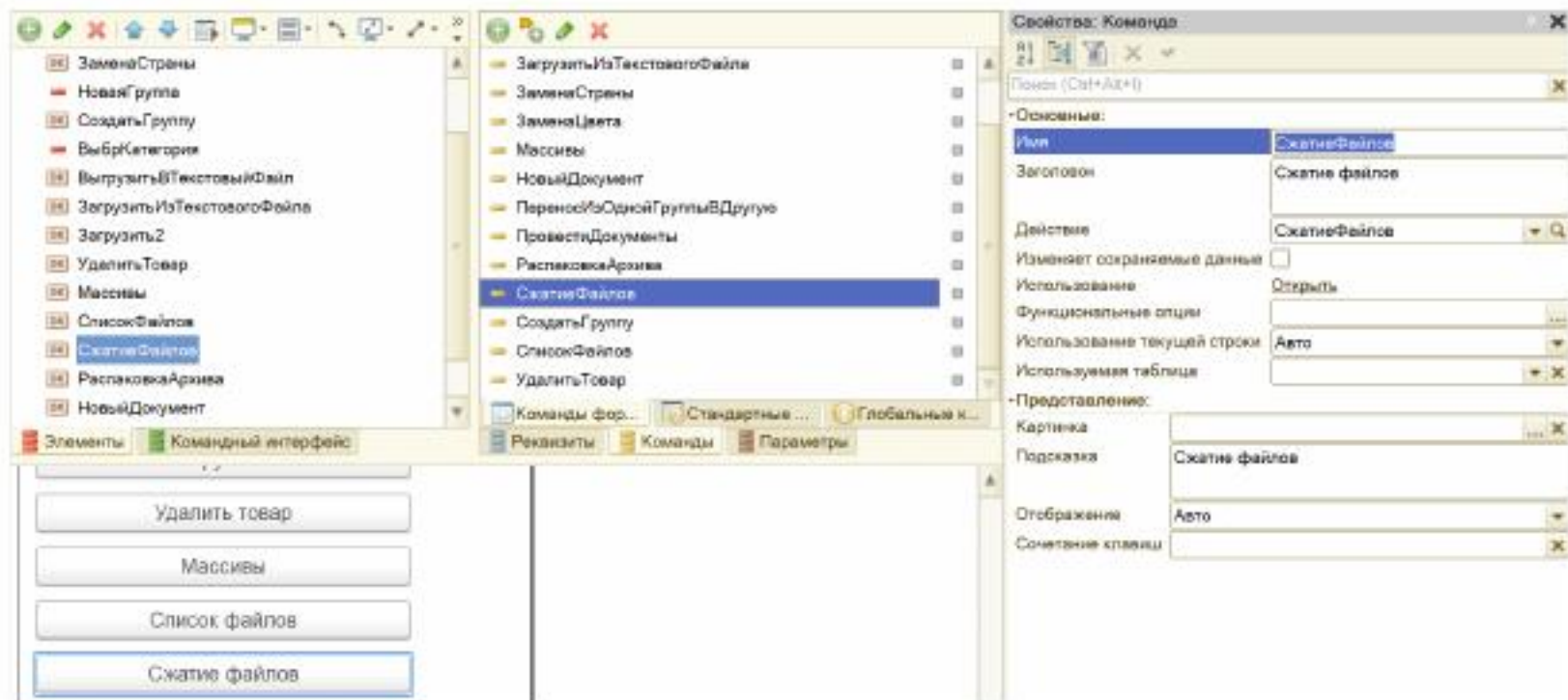


Рис. 26. Добавление команды «СжатиеФайлов»

Запрограммируем команду следующим образом.

&НаСервереБезКонтекста

Процедура СжатиеФайловНаСервере()

Архив = *Новый*
ЗаписьZipФайла("C:\Users\alexeysemenov\Desktop\Можно
удалить\1с.zip",,,МетодСжатияZip.Сжатие,
УровеньСжатияZip.Максимальный,);

Архив.Добавить("C:\Users\alexeysemenov\Desktop\Можно
удалить.txt",РежимСохраненияПутейZip.СохранятьОтносительныеПу*
ти,);

Архив.Записать();
ФайлНаДиске = *Новый*
Файл("C:\Users\alexeysemenov\Desktop\Можно удалить\1с.zip");

Если ФайлНаДиске.Существует() Тогда

Сообщить("Архив создан!");

Иначе

Сообщить("Архив не получилось создать!");

*КонецЕсли;
КонецПроцедуры*

&НаКлиенте

Процедура СжатиеФайлов(Команда)

СжатиеФайловНаСервере();

КонецПроцедуры

Здесь мы создаем новый объект с помощью *ЗаписьZipФайла*, который предназначен для записи в ZIP-архив. С помощью метода *Добавить* указываем, что именно мы хотим заархивировать. Далее записываем архив на диск и проверяем появился ли этот архив на диске.

Добавим на форму кнопку и проверим реализацию сжатия файлов в пользовательском режиме 1С (см. рис. 27). Действительно, обработка находит только файлы с расширением txt в указанной папке и запаковывает их в zip-архив.

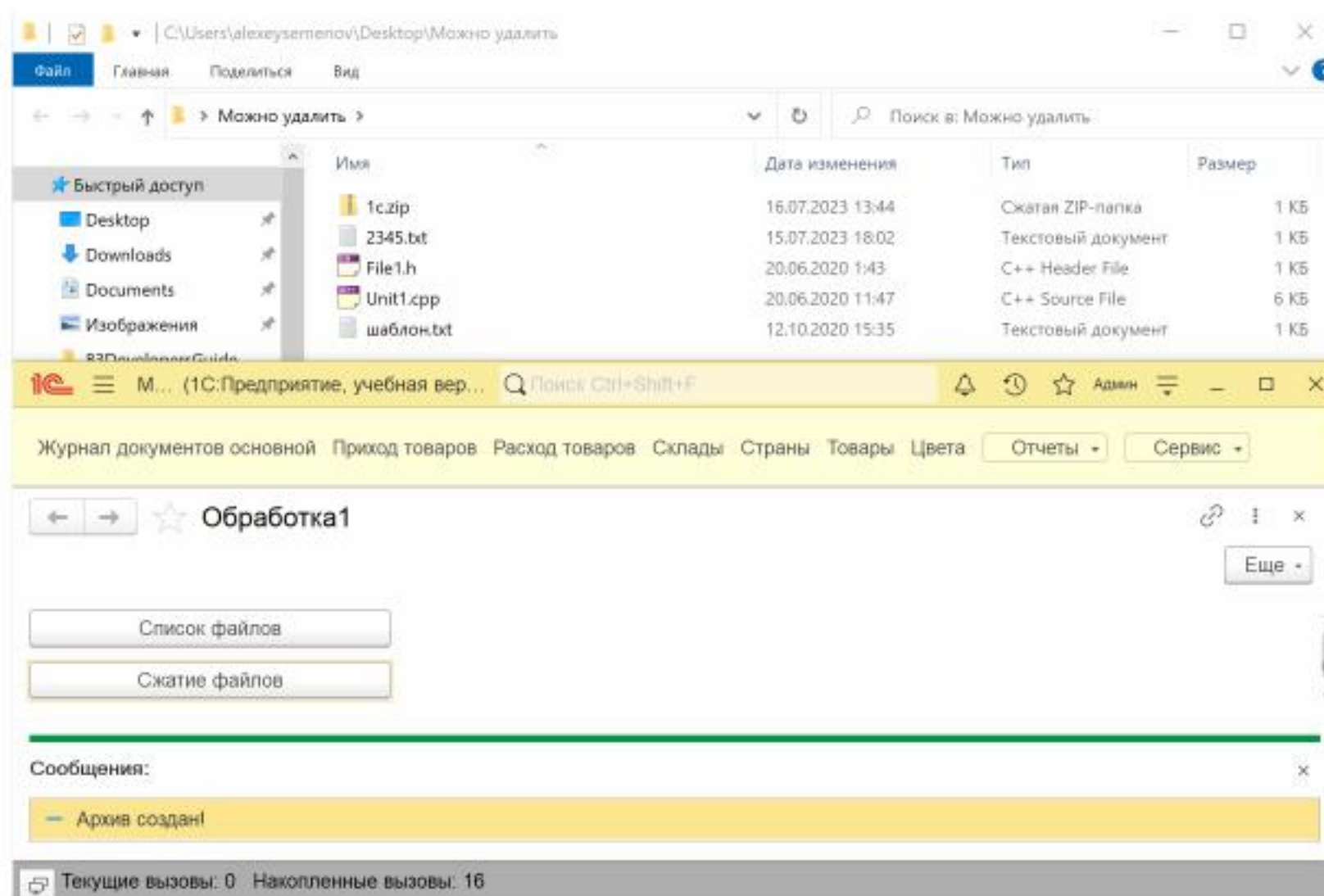


Рис. 27. Результат работы обработки «Сжатие файлов»

Создание команды «Распаковка архива»

Рассмотрим реализацию команды, позволяющей распаковать архив файлов. Как обычно, начнем с добавления новой команды (см. рис. 28).

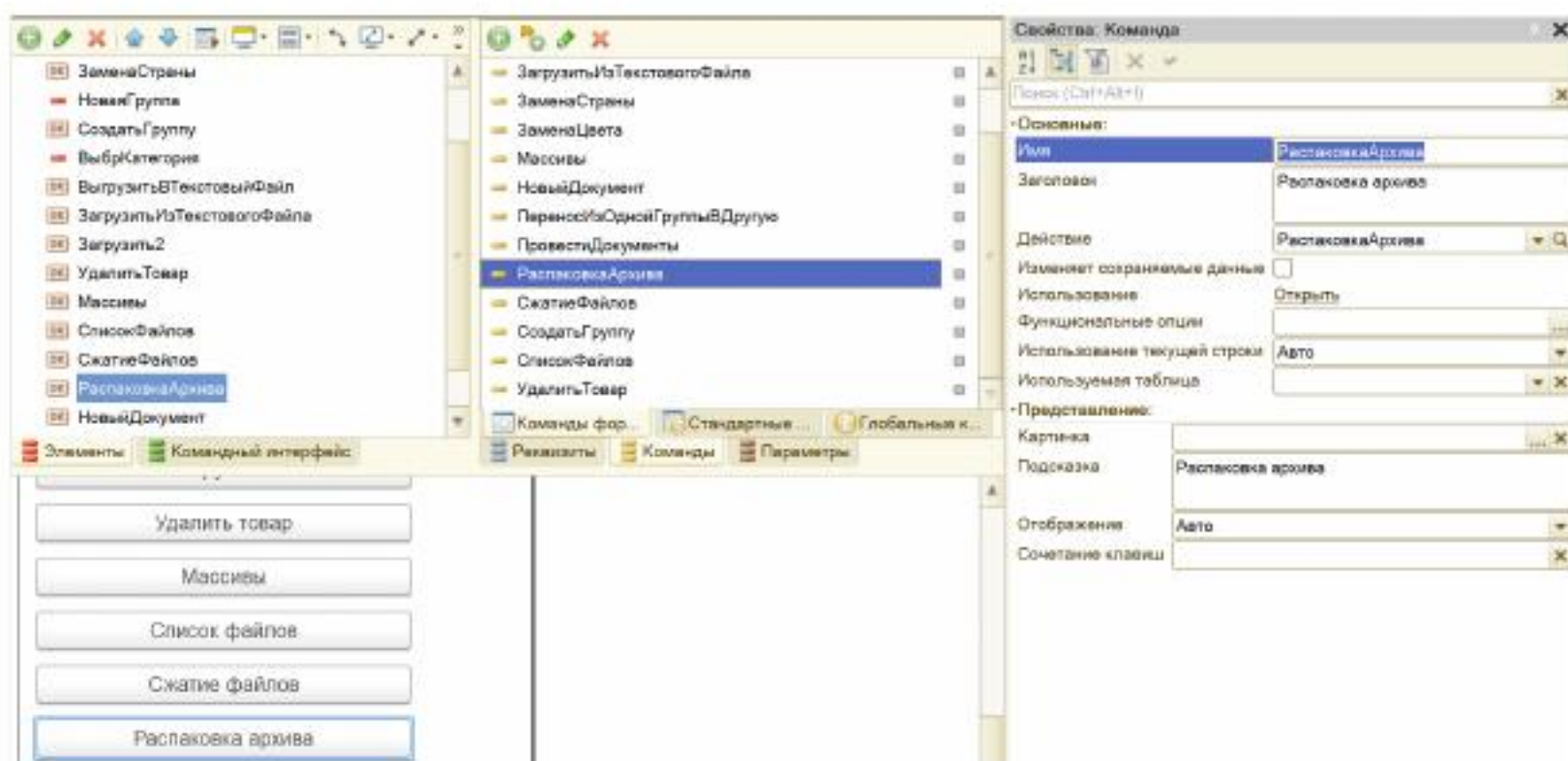


Рис. 28. Добавление команды «РаспаковкаАрхива»

Для реализации распаковки запрограммируем следующий код.

&НаСервереБезКонтекста

Процедура РаспаковкаАрхиваНаСервере()

ФайлНаДиске

=

Новый

Файл("C:\Users\alexeysemenov\Desktop\Можно удалить\1с.zip");

Если ФайлНаДиске.Существует() Тогда

Архив = Новый ЧтениеZipФайла(ФайлНаДиске.ПолноеИмя);

*Архив.ИзвлечьВсе("C:\Users\alexeysemenov\Desktop\Можно
удалить\1с_1с");*

Архив.Закрыть();

КонецЕсли;

КонецПроцедуры

&НаКлиенте

Процедура РаспаковкаАрхива(Команда)

РаспаковкаАрхиваНаСервере();

КонецПроцедуры

Логика здесь простая. Создаем объект для работы с файлом. Если нужный нам архив существует на диске, то используя метод *ИзвлечьВсе*, распаковываем его.

Создадим на форме кнопку и проверим реализацию распаковки архива файлов в пользовательском режиме 1С (см. рис. 29).

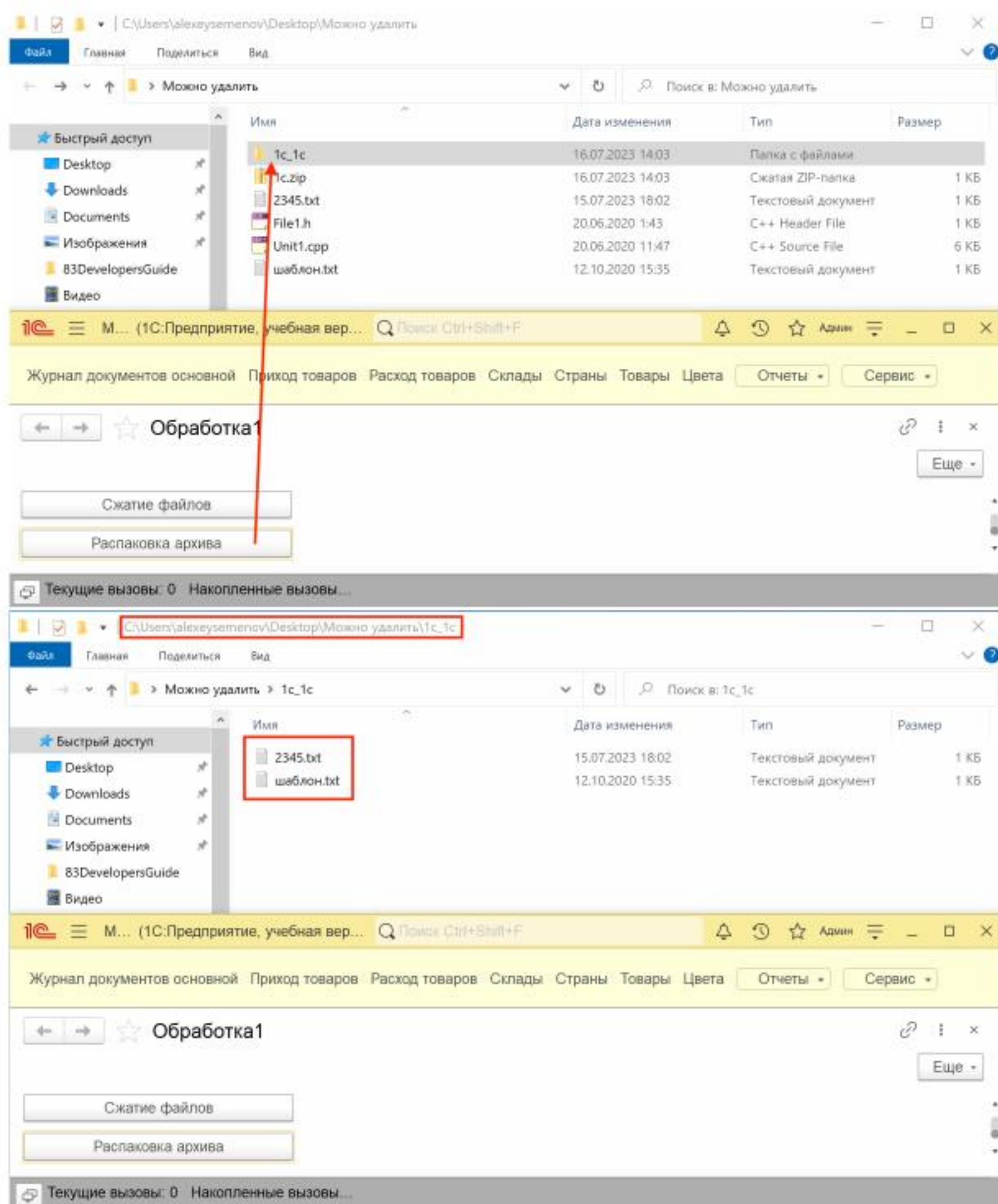


Рис. 29. Результат работы обработки «Распаковка архива»

Создание команды «Новый документ»

Рассмотрим реализацию команды, позволяющей создавать новый документ. Как обычно, начнем с добавления новой команды (см. рис. 30).

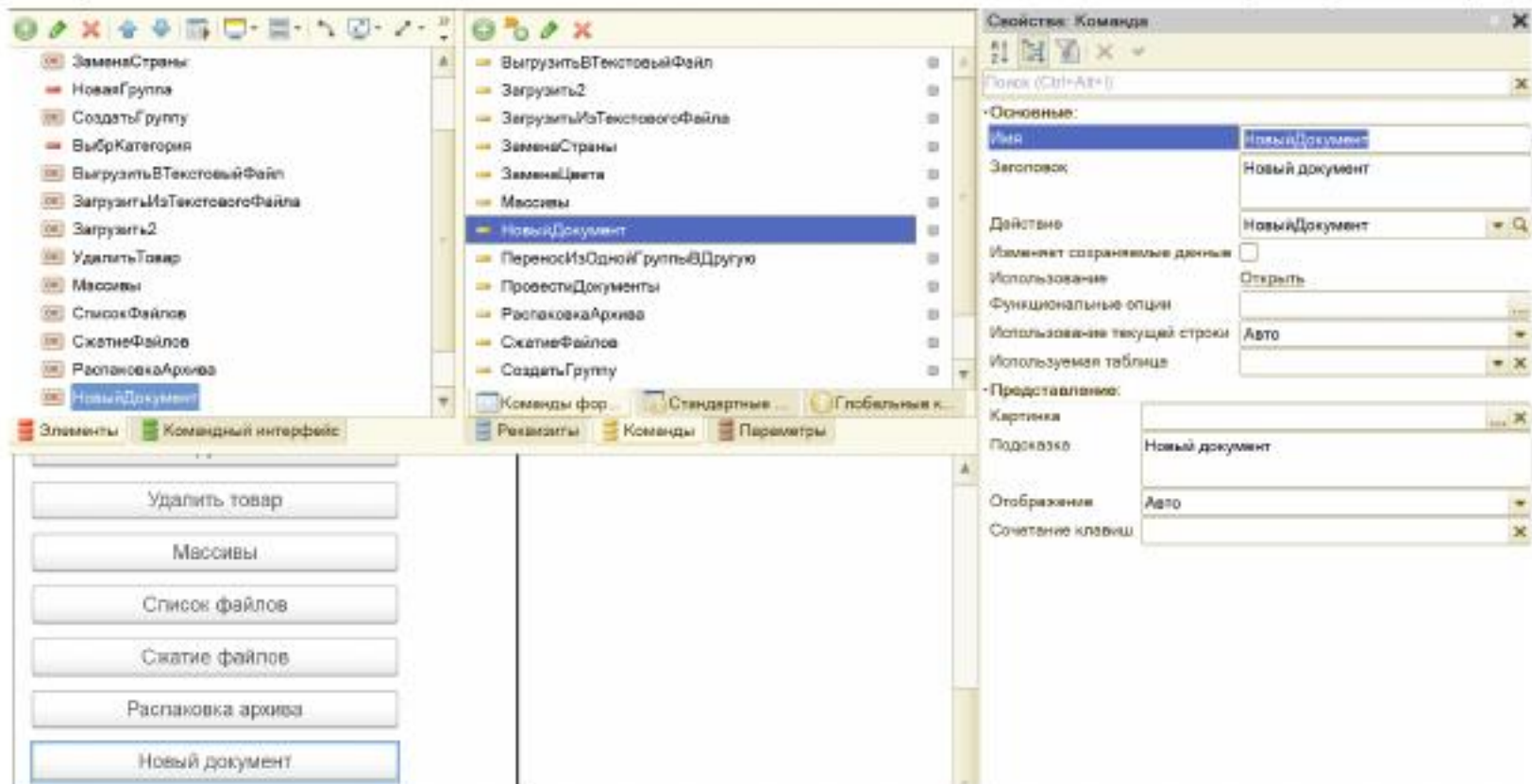


Рис. 30. Добавление команды «НовыйДокумент»

Приведем программный код, реализующий программное создание нового документа.

&НаСервереБезКонтекста

Процедура НовыйДокументНаСервере()

НовыйПриходТовара

=

Документы.ПриходТоваров.СоздатьДокумент();

НовыйПриходТовара.Дата = ТекущаяДата();

НужныйСклад

=

Справочники.Склады.НайтиПоНаименованию("Склад основной");

НовыйПриходТовара.Склад = НужныйСклад;

НоваяСтрока = НовыйПриходТовара.СписокТоваров.Добавить();

НужныйТовар

=

Справочники.Товары.НайтиПоНаименованию("Наушники");

НоваяСтрока.Товар = НужныйТовар;

НоваяСтрока.Количество = 30;

НовыйПриходТовара.Записать(РежимЗаписиДокумента.Проведение);

КонецПроцедуры

&НаКлиенте

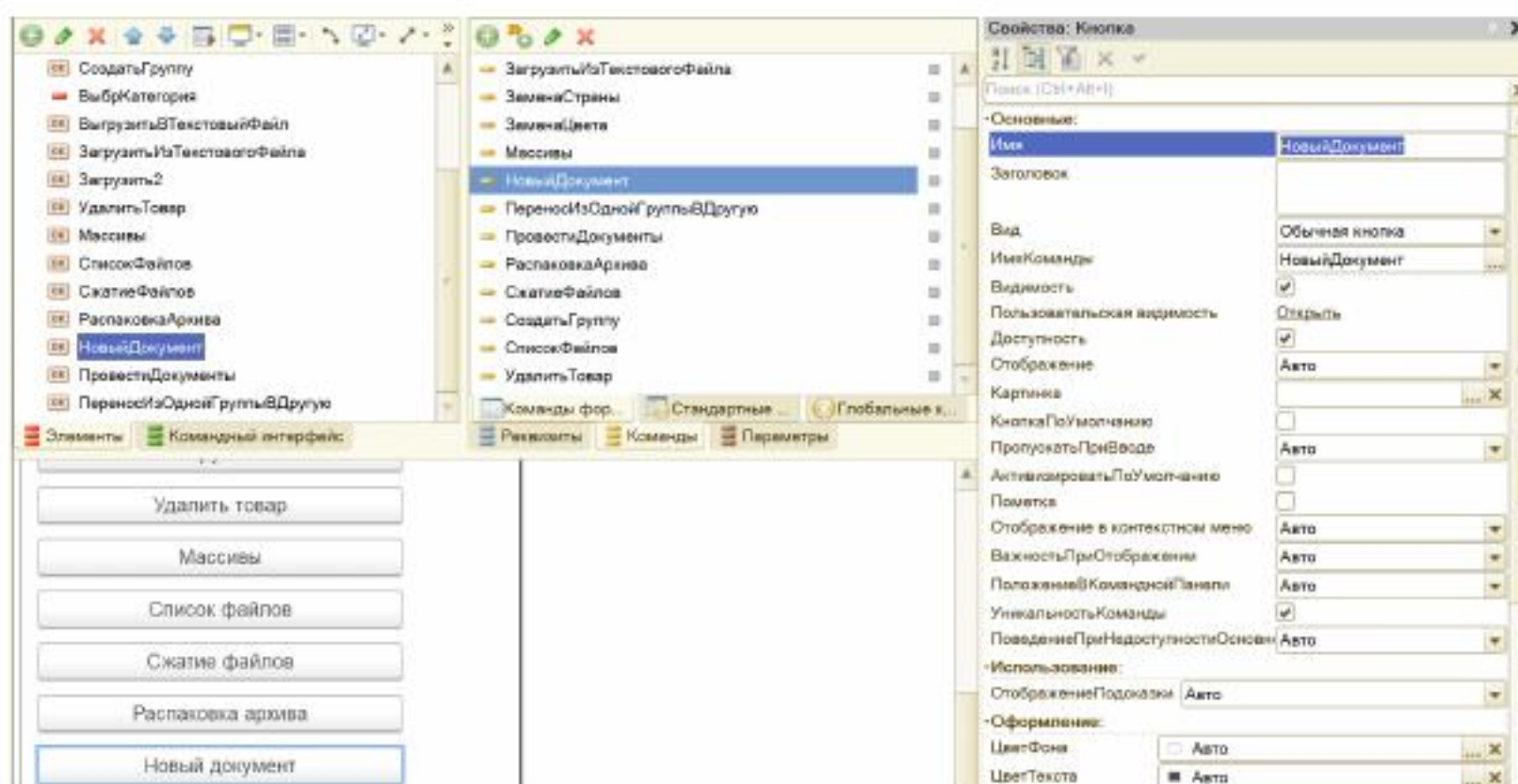
Процедура НовыйДокумент(Команда)

НовыйДокументНаСервере();

КонецПроцедуры

В данном коде реализована следующая логика. Создаем объект нового приходного документа с помощью функции *СоздатьДокумент()*. Далее заполняем значениями реквизиты этого документа: дата, склад, табличная часть «СписокТоваров». В табличную часть записываем только наименование товара и количество. Затем осуществляем запись документа с проведением.

Создадим на форме кнопку и проверим работу реализованного алгоритма в пользовательском режиме 1С (см. рис. 31).



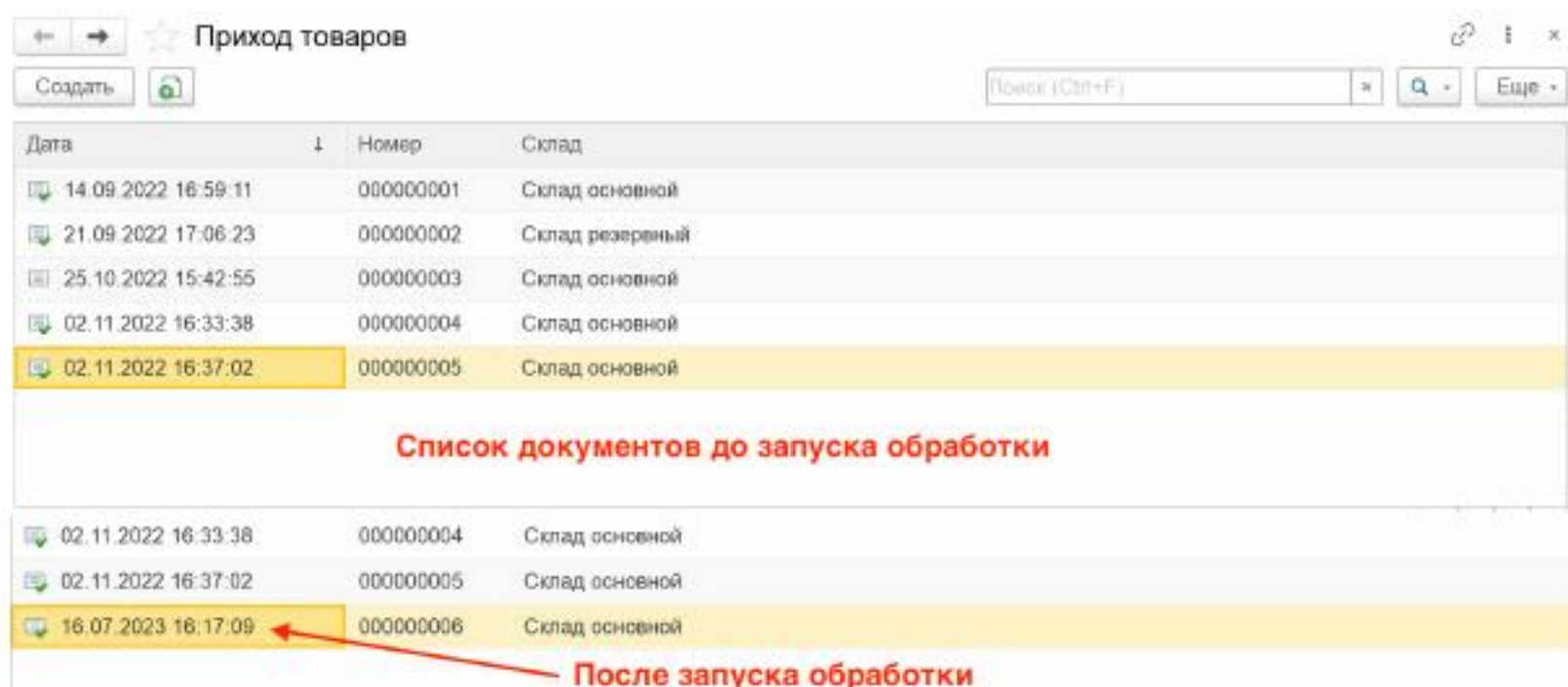


Рис. 31. Результат работы обработки «Новый документ»

Создание команды «Провести документы»

Рассмотрим реализацию команды, позволяющей осуществлять программное проведение документов. Как обычно, начнем с добавления новой команды (см. рис. 32).

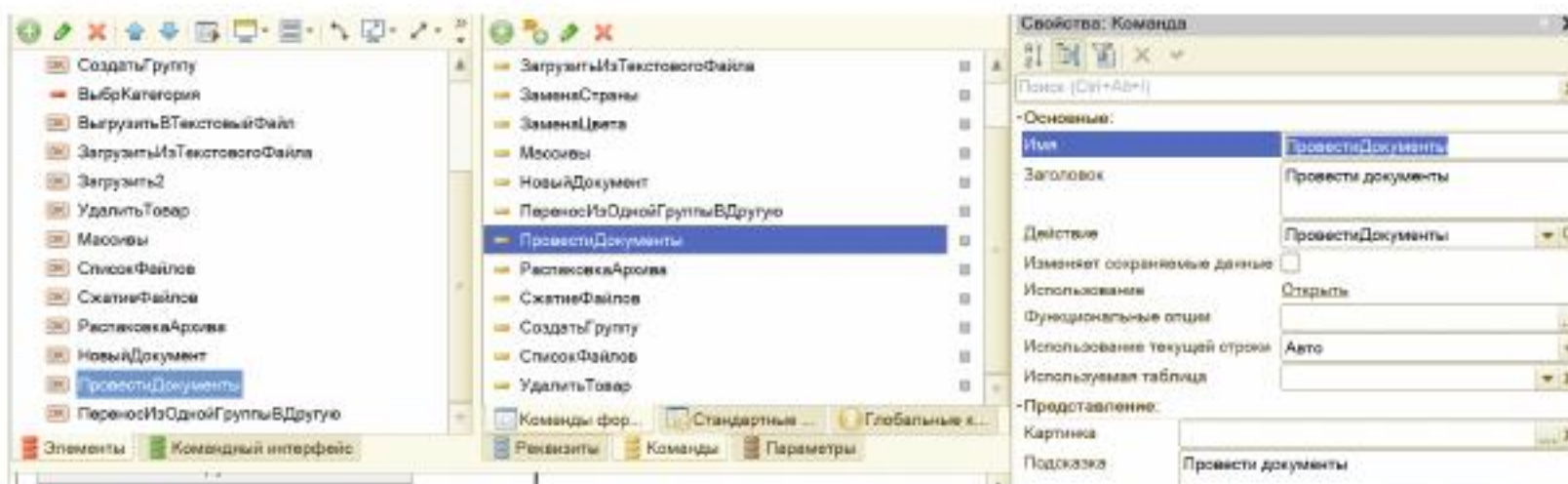


Рис. 32. Добавление команды «ПровестиДокументы»

Рассмотрим программный код, реализующий программное проведение документов.

&НаСервереБезКонтекста

Процедура ПровестиДокументыНаСервере()

ВыборкаДокументов

=

*Документы.РасходТоваров.Выбрать(НачалоГода(ТекущаяДата()),
КонецГода(ТекущаяДата()));*

Пока ВыборкаДокументов.Следующий() Цикл

```

ТекущийДокумент = ВыборкаДокументов.ПолучитьОбъект();
НомерДок=СокрЛП(ТекущийДокумент.Номер);
ДатаДок=СокрЛП(ТекущийДокумент.Дата);
СкладДок=СокрЛП(ТекущийДокумент.Склад);
Сообщить(НомерДок + ", " + ДатаДок + ", " + СкладДок);

ВсегоПоДок=ТекущийДокумент.СписокТоваров.Итог("Количество"
);
Сообщить("Кол-во по документу: " + ВсегоПоДок);
Если ТекущийДокумент.Проведен <> Истина Тогда
    Сообщить("Документ " + НомерДок + " не проведен!");
    Попытка
        Если
ТекущийДокумент.ПроверитьЗаполнение()=Истина Тогда ///!!

        ТекущийДокумент.Записать(РежимЗаписиДокумента.Проведение);
        Если ТекущийДокумент.Проведен = Истина
Тогда
            Сообщить("Документ " + НомерДок +
" проведен!");

            КонецЕсли;
        КонецЕсли;

        Исключение
            Сообщить("Невозможно провести Документ " +
НомерДок);
        КонецПопытки;

        КонецЕсли;
        Сообщить("-----");
    КонецЦикла;
КонецПроцедуры

&НаКлиенте
Процедура ПровестиДокументы(Команда)
    ПровестиДокументыНаСервере();

```


КонецПроцедуры

В данном коде реализована следующая логика. Помещаем в переменную *ВыборкаДокументов* все расходные документы, начиная с начала года и заканчивая концом года. Организуем цикл по нашей выборке до тех пор, пока в ней есть следующий элемент. Получаем объект текущего документа и считываем из него основные реквизиты (номер, дата, склад). Выводим их в сообщении пользователю. Далее рассчитываем итоговое количество по документу и выводим его в сообщении пользователю. Затем проверяем текущий документ на проведение. Если он не проведён, то выводим сообщение об этом и пытаемся его провести. Проверяем заполнен ли он с помощью метода *ПроверитьЗаполнение()*. Метод проверяет заполнение реквизитов в документе. Для реквизитов, у которых при конфигурировании свойство "Проверка заполнения" установлено в значение "Показывать ошибку" и реквизит не заполнен, будет сгенерировано сообщение об ошибке. В строке *ТекущийДокумент.Записать(РежимЗаписиДокумента.Проведение)*; записываем документ в режиме «Записать и провести». Еще раз после записи проверяем расходный документ на проведение. В остальных случаях вызывается исключение с сообщением о невозможности проведения документа.

Создадим на форме кнопку (см. рис. 33).

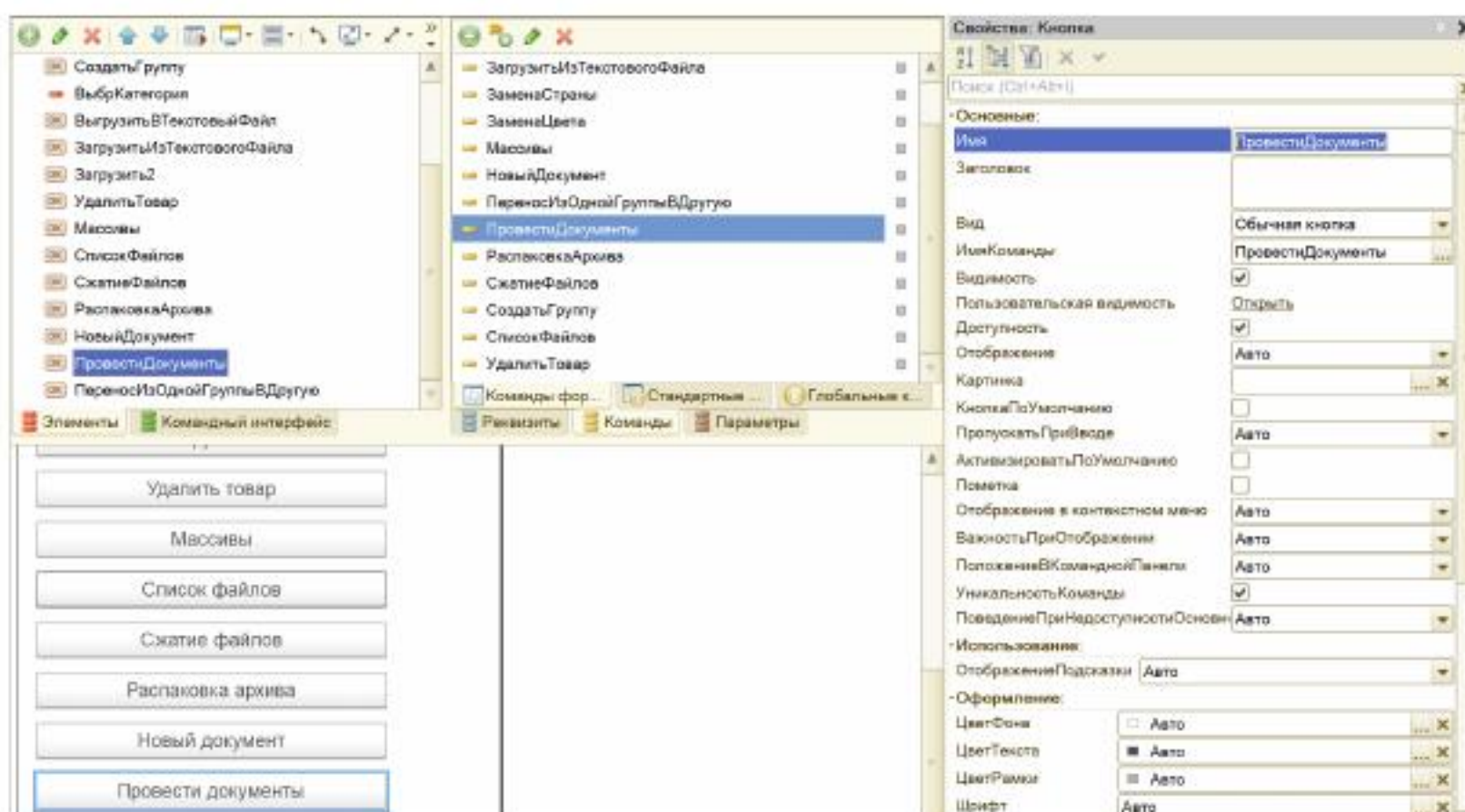


Рис. 33. Создание кнопки «Провести документы»

Проверим работоспособность реализованного алгоритма проведения документов в пользовательском режиме 1С. Для начала создадим явно ошибочный документ, в котором нет ни одного товара и не указан склад (см. рис. 34).

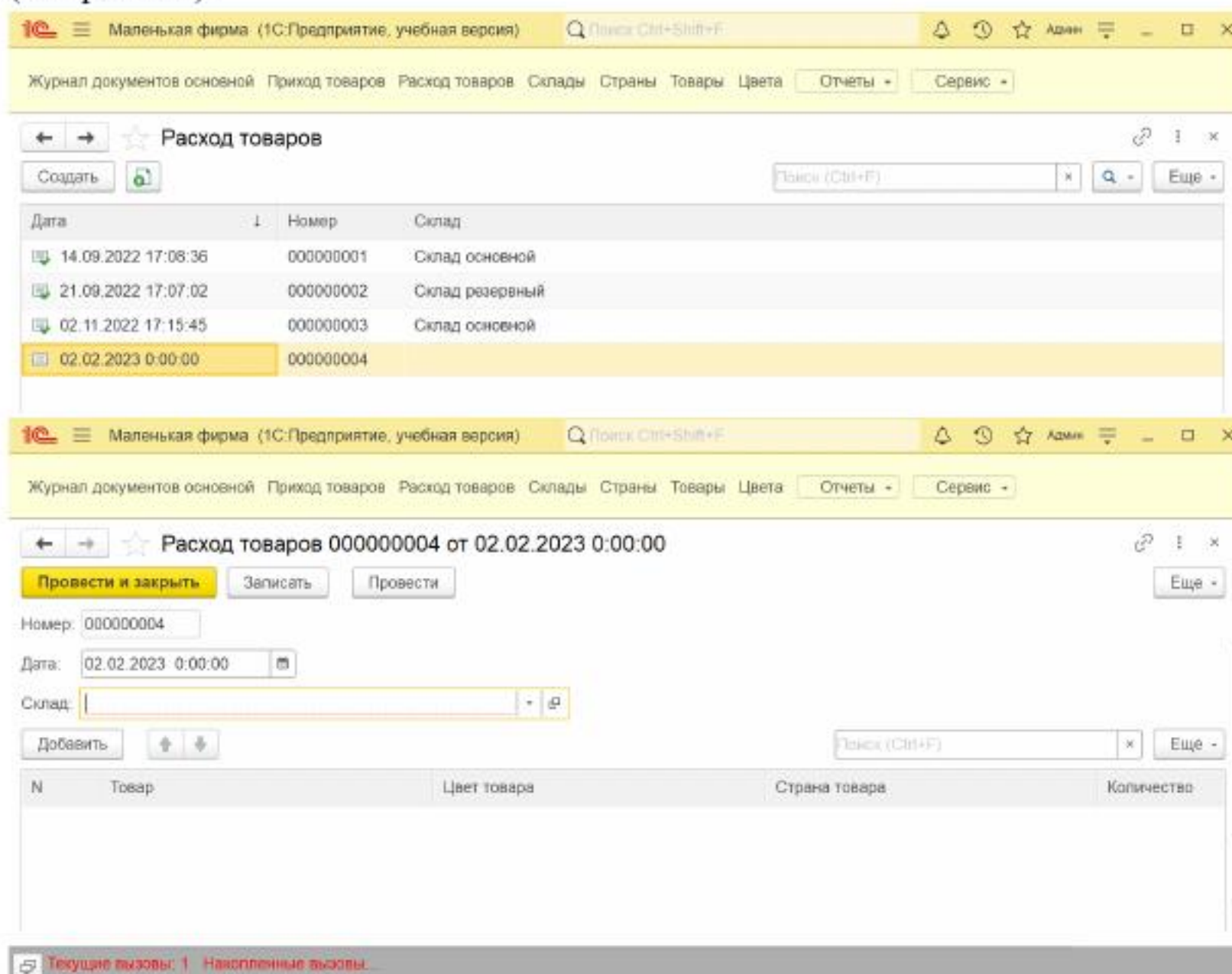


Рис. 34. Ошибочный и не проведённый расходный документ

Запускаем нашу обработку и видим, что документ не прошёлся и система выдала соответствующее сообщение (см. рис. 35). Вполне ожидаемо и подтверждает, что наш алгоритм работает правильно.

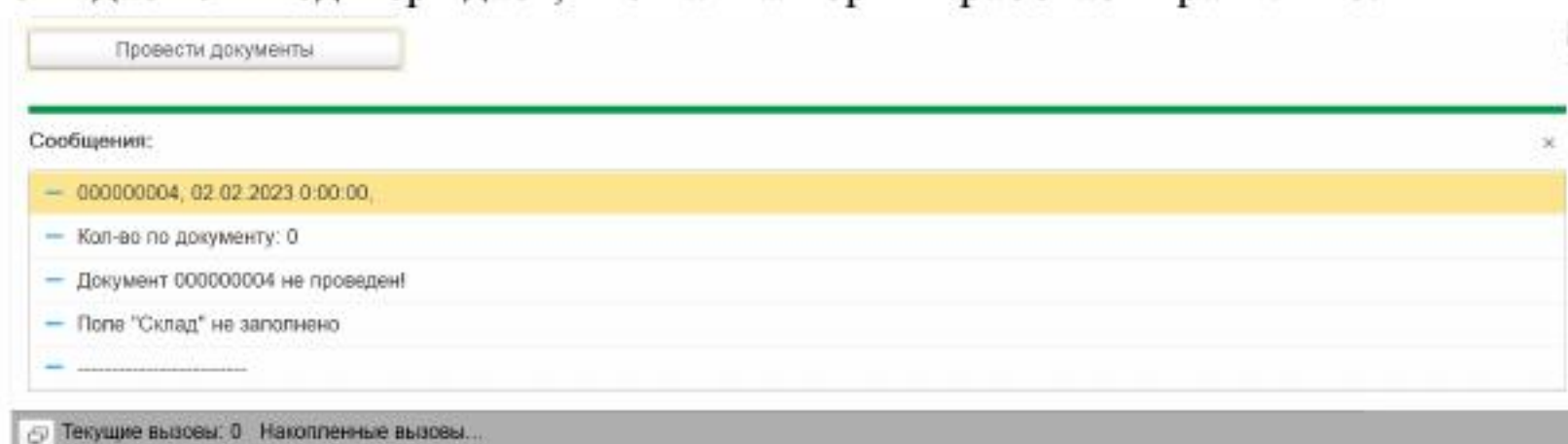


Рис. 35. Отказ в проведении документа

Теперь исправим ошибки в этом документе. Выберем склад и какие-нибудь товары, которые есть на остатках. Укажем количество в пределах остатков. Сохраним изменения, но без проведения (см. рис. 36).

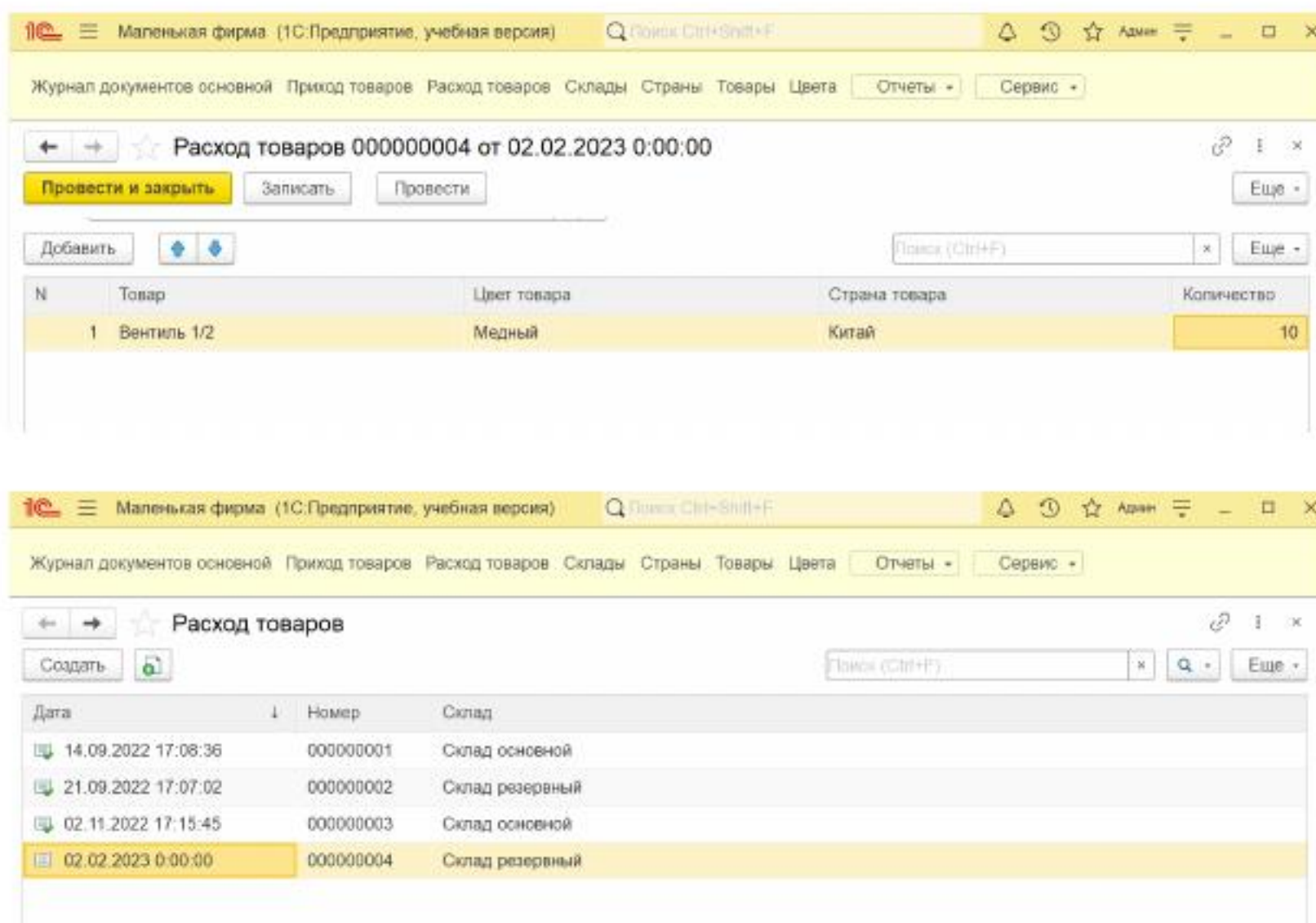
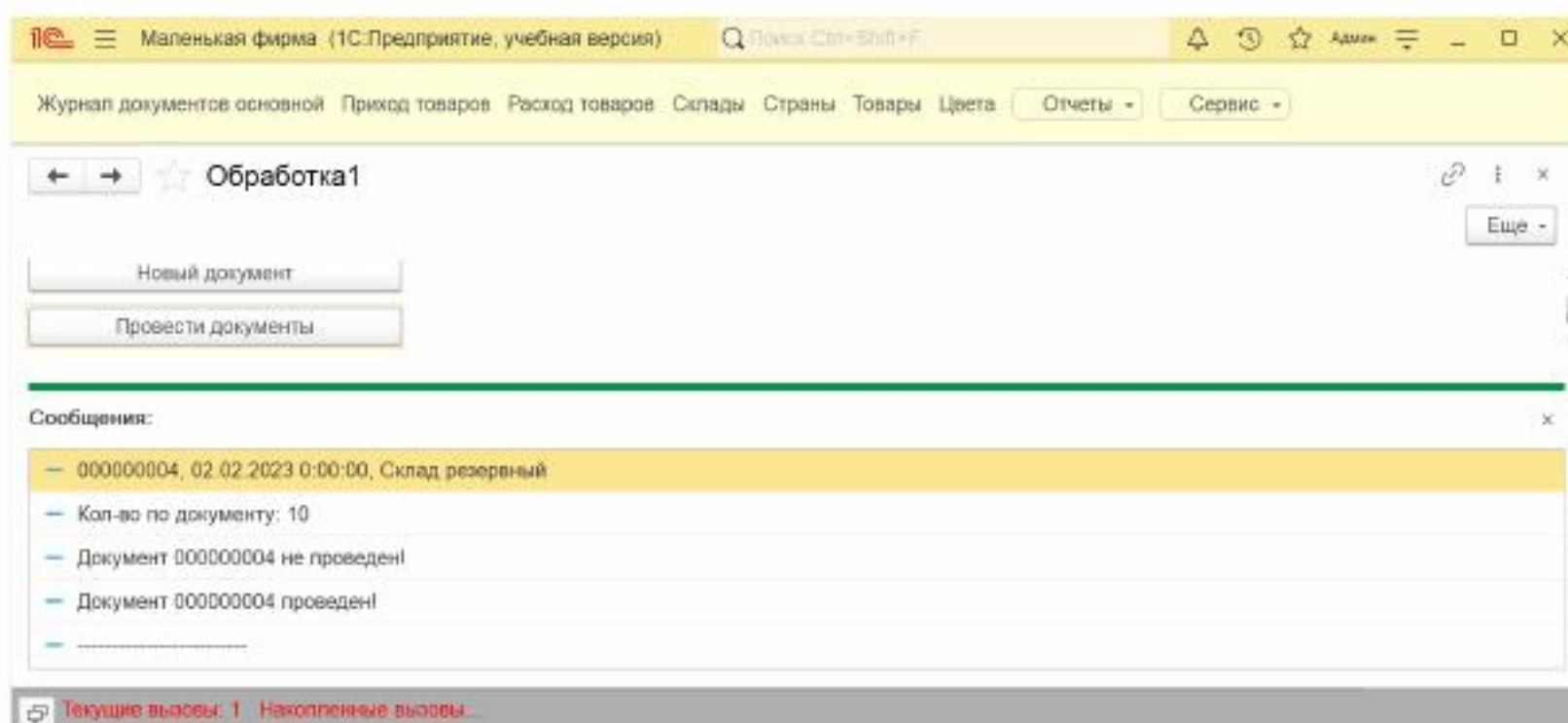


Рис. 36. Подготовленный для тестирования документ

Заново запустим нашу обработку и увидим сообщение об успешном проведении документа (см. рис. 37).



Дата	Номер	Склад
14.09.2022 17:08:36	000000001	Склад основной
21.09.2022 17:07:02	000000002	Склад резервный
02.11.2022 17:15:45	000000003	Склад основной
02.02.2023 0:00:00	000000004	Склад резервный

Рис. 37. Проведенный документ после запуска обработки

Теперь проверим ситуацию, когда в не проведенном документе указано количество товара, превышающее количество на остатках, например 100 штук. Отменим проведение документа, изменим количество в большую сторону и заново запустим нашу обработку. Система выдаст сообщение о невозможности проведения документа, и это абсолютно верно (см. рис. 38).

Провести и закрыть Записать Провести

Номер: 000000004

Дата: 02.02.2023 0:00:00

Склад: Склад резервный

N	Товар	Цвет товара	Страна товара	Количество
1	Вентиль 1/2	Медный	Китай	100

Провести документы

Сообщения:

- 000000004, 02.02.2023 0:00:00, Склад резервный
- Кол-во по документу: 100
- Документ 000000004 не проведен!
- Не хватает кол-ва для товара Вентиль 1/2, нужно еще 72
- Невозможно провести Документ 000000004

Текущие вызовы: 0 Накопленные вызовы:...

Рис. 38. Документ не должен провестись, и он не провёлся

Создание команды «Перенос из одной группы в другую»

Рассмотрим реализацию команды, позволяющей перенести товары из одной категории в другую. По аналогии с предыдущими примерами добавим новую команду и реализуем следующий программный код.

&НаСервереБезКонтекста

Процедура ПереносИзОднойГруппыВДругуюНаСервере()

Запрос = Новый Запрос;

Запрос.Текст = "ВЫБРАТЬ

| Товары.Ссылка

| ИЗ

| Справочник.Товары КАК Товары

| ГДЕ

| Товары.Родитель = &Родитель";

Товары = Справочники.Товары;

НужнаяГруппаТоваров

=

Товары.НайтиПоНаименованию("Сантехника");

Запрос.УстановитьПараметр("Родитель", НужнаяГруппаТоваров);

Результат = Запрос.Выполнить();

Выборка = Результат.Выбрать();

НоваяГруппаТоваров = Товары.НайтиПоНаименованию("Лаки и краски");

Пока Выборка.Следующий() Цикл

ТекущийТовар = Выборка.Ссылка.ПолучитьОбъект();

ТекущийТовар.Родитель = НоваяГруппаТоваров;

ТекущийТовар.Записать();

Сообщить("Товар " + ТекущийТовар.Наименование + " перенесен в новую категорию " + СокрЛП(НоваяГруппаТоваров));

КонецЦикла;

КонецПроцедуры

&НаКлиенте

Процедура ПереносИзОднойГруппыВДругую(Команда)

ПереносИзОднойГруппыВДругуюНаСервере();

КонецПроцедуры

Список литературы

1. Радченко М.Г., Хрусталева Е.Ю. 1С:Предприятие 8.3. Практическое пособие разработчика. Примеры и типовые приемы. –М.: ООО «1С-Паблишинг», 2013. 964 с.
2. Байдаков В., Дранищев В., Королькова Е., Краюшкин А., Кузнецов И., Лавров М., Моничев А., Плякин А., Радченко М. 1С:Предприятие 8.3. Руководство разработчика. –М.: ООО «1С», 2016.
3. Семенов А.А. Разработка приложений 1С:Предприятие для обувной промышленности: методические указания. – М.: МГУДТ, 2014. 31 с.
4. Семенов А.А., Муртазина А.Р. Разработка специализированных конфигураций 1С: методические указания. – М.: МГУДТ, 2015. 37 с.
5. Семенов А.А., Муртазина А.Р. Программирование в среде 1С:Предприятие. Часть 1: учебно-методическое пособие. – М.: РГУ им. А.Н. Косыгина, 2017. 36 с.
6. Семенов А.А., Муртазина А.Р. Программирование в среде 1С:Предприятие. Часть 2: учебно-методическое пособие. – М.: РГУ им. А.Н. Косыгина, 2018. 36 с.
7. Семенов А.А. Разработка на платформе 1С:Предприятие 8.3. Часть 1: Учебное пособие – М.: ФГБОУ ВО «РГУ им. А.Н. Косыгина», 2023. – 48 с.
8. Архитектура платформы 1С:Предприятие: [Электронный ресурс]. URL: <https://v8.1c.ru/platforma/obrabotka/> (дата обращения 05.12.2023)

Учебная литература

Семенов Алексей Александрович

**РАЗРАБОТКА НА ПЛАТФОРМЕ
1С:ПРЕДПРИЯТИЕ 8.3. ЧАСТЬ 2**

Учебное пособие

Усл.печ.л. _____ Тираж 10 экз. Заказ № _____

Редакционно-издательский отдел ФГБОУ ВО «РГУ им. А.Н. Косыгина»

115035, Москва, ул. Садовническая, 33, стр.1

тел. 8-495-811-01-01 доб. 1099

e-mail: riomgudt@mail.ru

Отпечатано в РИО ФГБОУ ВО «РГУ им. А.Н. Косыгина»